

Book Notes & To-Do

A running scratchpad for the eventual book adaptation of this course

VIDEO GAME CREATION

How To Use This Document

This is a working scratchpad, not a polished outline - it exists so that adding a thought while the course is being built doesn't require deciding, right then, where it "really" belongs. Two rules keep it useful instead of becoming a mess:

- Add freely. A new To-Do or Note can sit next to, or even contradict, something written earlier - resolve the contradiction later, not in the moment. Rethinking the whole document every time something gets added defeats the point of having it.
- Reorganize rarely, on purpose. Periodic dedicated consolidation passes (see the standing To-Do item below) are where duplicates get merged, stale items get cut, and contradictions actually get resolved - not as a side effect of adding one new line.
- To-Do items get updated or deleted outright once actually resolved - this section should always reflect current, real open work, not a historical log. Notes & Commentary is the opposite: it's allowed to accumulate, and old entries are kept (even superseded ones) until a consolidation pass deliberately prunes them.

To-Do

Open items, pulled from the project's task list and from things noticed while building other documents. Check one off or delete it outright once it's actually done - don't leave resolved items sitting here.

- ☐ Book version: move the Godot signals sidebar ("Godot Signals: Create It, Connect It, Write the Function, Emit It", with the four-piece diagram_godot_signal_four_pieces) from the Lesson 8 Sidebars to Lesson 6, where signals first appear (the destroyed signal for hits), and remove it from Lesson 8. Lesson 8 should then only point back to it, since escaped is the second use of the same four pieces. For the current course the sidebar stays in the Lesson 8 Sidebars document, since no Lesson 6 Sidebars document exists yet. Source: topic in build_sidebars.js (lesson08 array). *(decision by Robert; book retrofit only)*
- ☐ Rule 15 (explicit, beginner-friendly names) retrofit: Godot's Label-node names were renamed this session (WinLabel/LoseLabel/ScoreLabel/LivesLabel/WaveLabel -> WinText/LoseText/ScoreText/LivesText/WaveText, matching GDevelop's existing text

object names) after a student read WinLabel as Window Label. Audit every other engine and lesson for names that lean on engine jargon (Label, Node, Actor, Pawn, Controller, Template, Trigger, Behavior, Instance) or that a newcomer could misread, rename them to plain names, and keep the same plain name in every engine per Rule 12. Update the Data Dictionary and Cross-Engine Consistency Reference to match. Also check Robert's own test projects, which still use the old names. *(Standards Rule 15, added per Robert's direction; Godot text-node names done in Lessons 6-14, nothing else audited yet)*

- ☐ Script/Visual parity cleanup for the book: for engines that offer both a scripting and a visual path (GameMaker GML Code vs. GML Visual, Unity C# vs. Visual Scripting, Unreal C++ vs. Blueprint), make sure the two paths produce the SAME gameplay and appearance - same positions, sizes, text, colors, timing, sounds, and variable names - so a student can switch paths without the game changing. Robert's own GameMaker Code and Visual projects have drifted apart through experimentation, and the drift is easy to repeat whenever a function call and a block do not map one-to-one or the options are not obvious. Do a side-by-side pass lesson by lesson: list every function call next to its equivalent block with identical field values, flag where an exact equivalent does not exist and say so explicitly (Rule 4/7), and re-check after every content change. Consider a short parity checklist per lesson in the Prep Checklist, and verify against Robert's actual test projects, not just the documents. *(raised by Robert; GameMaker is the known problem case)*
- ☐ Book retrofit naming decision (Robert): all asset/object names in the book use PascalCase with NO common prefixes or suffixes (no obj_, spr_, snd_, sfx_ and the like), e.g. GameManager, Enemy, PlayerShip. Assets are kept organized with each engine's own mechanisms instead - folders in GameMaker's and Unity's asset browsers, groups, Scenes, Content Browser folders, and so on - so related assets stay together without name prefixes. This applies to the book retrofit only: for the current course pass, existing variable-naming conventions for each engine stay as already established, and the shipped GameMaker content (obj_gamemanager and friends) is left alone. When the book retrofit happens, apply it in one pass across all engines, update Rule 12 and the Data Dictionary, and decide whether Lesson 2 setup should teach creating the asset folders. *(decision by Robert; supersedes the earlier separate asset-prefix and capitalization To-Dos)*
- ☐ Final deliverable for BOTH the course and the book: a Complete Script Document per engine and coding style - the entire finished Semester 1 project in one place (every object, script, event, function, graph, or block sequence in full, organized by object in build order, with the lesson that introduced each piece noted). Generate it from the same lesson build scripts so it cannot drift. Needed because lessons now show only the methods they add or modify (Standards Rule 14). SCOPE NOW FINALIZED as a live-course Lesson 15 deliverable (see the dedicated ToDo item below for the full decision and its explicit dependency on reviewing the remaining lessons first) - this entry now serves mainly as the original decision record; see the newer entry for the actual build plan. *(decision by Robert; Standards doc now lists it as a Final Deliverable; superseded in scope by the Lesson 15 Complete Codebase ToDo below)*
- ☐ Retrofit Standards Rule 14 (code completeness by level: Handout Level 1 complete code incl. initialization and display; Handout Levels 2-3 prose + diagrams; Teacher's Key complete code for all three levels) across Lessons 2-15 and all 8 engines. Lesson 8 (all 8 engines) and Lesson 13 (all 8 engines, Key only) are the cases done so far. Known weak spot: most other Keys describe Level 3 in talking-point prose only (or, as Lesson 13 turned out to have, no code block at all), and some Handouts show only the new lines. Audit lesson by lesson, starting with Lesson 9. Scope note: Rule 14 requires COMPLETE code for every method a lesson adds or modifies (whole methods, not just

the new lines), with the new lines highlighted; unchanged methods are not repeated, since the Complete Script Document (separate To-Do) covers the whole project, because showing only new code has already confused students. Lessons 8-15 are brought up to this standard as each is reviewed; Lessons 2-7 are deliberately deferred to the book retrofit (they currently show new code only), and Lessons 16-30 must follow it as authored. *(Standards Rule 14, added per Robert's direction; Lesson 8 and Lesson 13 (Key) now complete; Lessons 2-7 deferred to the book retrofit)*

- ☐ Finish the GameMaker __dnd_lives / __dnd_score review for Lessons 10-15 (restart/lives reset and any other lives/score use): replace deprecated built-in lives/score and "Set Instance Variable ... Lives/Score" with the purpose-built Lives/Score blocks (Target = obj_gamemanager) in Visual paths and __dnd_lives/__dnd_score in code paths. Deliberately held until Lessons 8 and 9 are fully reviewed. *(added after Lessons 8-9 GameMaker corrections; Lessons 10-15 not yet reviewed)*
- ☐ GameMaker: review every use of obj_name.variable notation in code paths and Visual paths. Where the script runs on the target object itself (or a block has a Target drop-down), prefer the plain variable name plus the Target drop-down over the dotted form, and make the code path and Visual path teach the same idea. Needs a deliberate pass across all lessons, not piecemeal edits. *(raised by Robert while reviewing Lessons 8-9)*
- ☐ GameMaker sidebar and comparison test: confirm that (a) the If Lives block with Target = GameManager, (b) an If Variable/Expression block testing __dnd_lives with Target = GameManager, and (c) a code if-statement using obj_gamemanager.__dnd_lives all behave identically, and write it up as a Sidebar (three ways to ask the same question about the same instance variable). Run the test in GameMaker first and record actual results before writing. *(raised by Robert; needs hands-on GameMaker testing)*
- ☐ Cross-engine design drift (HUD object, prefabs, helper objects): during the Lesson 10 review Robert noticed that structural pieces introduced in one engine are missing or different in others. Examples: Unity and Unreal got a dedicated HUD (a HUD GameObject / WBP_HUD widget) while Scratch, Godot, GameMaker, GDevelop, TIC-80 and 001 Game Creator draw score and lives directly from the GameManager or from labels; Unity uses Prefabs for enemies while GameMaker's equivalent was never used even though one exists. Before the book, audit every engine for pieces present in some but not all (HUD, prefab/template for enemies/bullets, spawner, restart routine, start_game function), decide per piece whether it is a course-wide concept (then add it to every engine, using each engine's native equivalent) or an engine-specific need (then document the driving reason per Rule 5), and record the result in the Cross-Engine Consistency Reference and Data Dictionary. Add a Standards check: any new structural object introduced for one engine must be introduced for all, or the reason documented. *(raised by Robert at Lesson 10 review; audit needed)*
- ☐ Cross-engine name normalization (generalized): the same object, variable and function must be spelled identically in every engine and every document wherever the engine allows, and the casing must follow the single book convention (PascalCase for assets and objects, per the naming decision above). Known example found at the Lesson 10 review: GameMaker's GameManager object is spelled obj_gamemanager in the shipped handouts but obj_GameManager in Robert's own test project (drift from the late switch to PascalCase). In the book pass, audit and normalize every name (GameManager, game_active, enemies_remaining, lives, score, start_game and the rest) against the Data Dictionary, per engine, then regenerate all Handouts, Keys, Slides, diagrams and the Complete Script Documents from the normalized names. *(raised by Robert at Lesson 10 review; book pass)*

- ☐ Canonical bug-hunt exercise (decision): the win/lose gap (kills reduce enemies_remaining but escapes only reduce lives, so some rounds end with no win and no loss, and R does nothing) is INTENTIONALLY left in Lessons 6 and 8. It is the Lesson 10 Level 3 'Fix a real bug!' task and part of the canonical lesson plan, also for the book. Do not fix it earlier. Plan at least one similar find-and-fix-your-own-design-bug exercise in Semester 2 (Lessons 16-30), where the groundwork is laid in earlier lessons by thinking about two cases separately instead of as one state. *(decision by Robert at Lesson 10 review)*
- ☐ Reconsider static members in Unity's GameManager (Robert): from Lesson 6 on, the Unity GameManager keeps shared state in static fields and static methods (RegisterKill, RegisterEscape, score, lives, gameActive). That is more complicated than the other engines need and forced a Singleton in the first Lesson 10 draft, since a static method cannot use an Inspector reference. Lesson 10 now avoids it with an ordinary instance method ResetGame() plus an Inspector slot for the spawner. For the book, decide whether GameManager should be a normal component with instance fields (other scripts reach it through an Inspector reference, as with the enemy prefab), and retrofit Lessons 6-10. Static members and singletons may then be a good Semester 2 refactoring topic, introduced on purpose. *(raised by Robert at Lesson 10 review; book pass)*
- ☐ Unity GameManager without statics AND without Find...() (book design change, related to the static-members item above): in the book version, avoid FindFirstObjectByType and similar lookups as well as statics. The obstacle is that a runtime-spawned prefab (Enemy, Bullet) cannot hold an Inspector reference to a scene object. Robert believes he solved this in an earlier project: REVIEW HIS OLD PROJECTS for a beginner-level approach before choosing one. Candidates to compare against it: (1) the spawner holds the GameManager reference in its own Inspector slot and hands it to each new enemy right after Instantiate; (2) the GameManager does the spawning itself, so it already holds every reference; (3) events/UnityEvents, the Unity cousin of the Godot signals sidebar (create it, connect it at spawn, handle it on GameManager, invoke it); (4) a shared ScriptableObject holding the game state. Pick the one with the least new vocabulary that fits Lessons 6-10, record the decision, and update Rule 12 and the Data Dictionary so the Unity design matches the other engines. *(raised by Robert at Lesson 10 review; review old projects first)*
- ☐ Design principle, reiterated (cross-engine drift): the game design should mirror itself from engine to engine as much as possible: same objects, same names, same state on the same manager, same flow, same behavior on screen. The one exception is where an engine fundamentally refuses to do it, or where mirroring would need an excessive, unnecessary workaround. In that case the engine's own idiom overrules consistency, and the deviation is recorded with its driving reason (Standards Rules 5 and 12). Apply this test to every item in the drift family: the HUD object, prefabs, the Unity statics and Find...() question, Godot signals versus other patterns, and the Scratch clone and broadcast handling. When one engine needs a heavier pattern than the others (the Lesson 10 Unity Singleton was the example), look first for a simpler design that all engines can share before accepting the deviation. During the book pass, fold this wording into Rule 12 and keep a per-engine list of accepted idiom exceptions in the Cross-Engine Consistency Reference and Data Dictionary. *(raised by Robert at Lesson 10 review; principle for the book pass)*
- ☐ Objects talking to each other: signals, broadcasts and events (add to the Unity design reconsideration above): Godot signals have caused difficulty and blank stares, and the course never gave a proper discussion of what a signal IS before using one. Scratch's Broadcast was glossed over the same way, and Unity events (candidate 3 above) would

raise the same problem. For the book, treat 'one object announces something, another object reacts' as a named concept introduced on purpose, before it is first used (Lesson 6 or earlier, matching the signals-sidebar move already logged), with the same vocabulary and the same picture across engines: Scratch broadcast and receive, Godot signal and connect, Unity event, GameMaker direct calls. Also compare simpler Godot alternatives before settling on signals: the Enemy calls a GameManager function directly through a node path, a Group call (get_tree().call_group), or an Autoload GameManager (Godot's counterpart of the static idea above, so decide it together with the Unity static question). Robert is not sure Godot has a simpler alternative, so verify these against a real project. Pick one pattern per engine, record the reasons, and add the picture to the Rule 9 diagram set. *(raised by Robert at Lesson 10 review; design decision plus a new concept lesson)*

- ☐ Unity static GameManager - revisit in THIS course if needed: the Lesson 6-12 Unity code keeps its static GameManager for now. If a student struggles with it, add a 'Non-static variant' box to the Unity Keys: remove static from the fields and from RegisterKill/RegisterEscape, and in every script that uses GameManager (Enemy.cs, Bullet.cs, the Lesson 12 sound code and so on) add a GameManager field filled in Start() with FindFirstObjectByType<GameManager>(), then change each GameManager.something call to use that field. Watch for the errors 'An object reference is required for a non-static member' and NullReferenceException. *(decision by Robert at Lesson 10 review; do only if needed)*
- ☐ Active-engine scope (set at the Lesson 10 review): the only engines students are currently using are Scratch, Godot, GameMaker (script and Visual) and Unity (script). Until the book pass, new and revised lesson work covers only those. Deferred, not dropped: Unreal, 001 Game Creator, Unity Visual Scripting, and GDevelop and TIC-80 (confirmed paused by Robert). For the book pass, catch each deferred engine up on every Rule 11-15 change, the Lesson 10 restart design, and the Rule 9 diagrams, using the shipped Lesson 10 content (Unreal and 001 Game Creator were already updated in Lesson 10 before this decision) as the starting point. Rendering and re-shipping the deferred engines' Handouts/Keys is paused until then. *(raised by Robert at Lesson 10 review; book pass)*
- ☐ Unity Visual Scripting pass (deferred): no students are currently using the Unity Visual Scripting path, so it was left out of the Lesson 10 pass. For the book, review every Unity Visual section from Lesson 8 on: verify node names against a real Unity install (e.g. the Lesson 10 restart graph and its On Input System Event Button node), apply the Rule 13/14 changes (state on the manager, complete code or graph per level), and build the missing Rule 9 Unity Visual node diagram for Lesson 10 (Lessons 7-9 already have one). *(raised by Robert at Lesson 10 review; book pass)*
- ☐ Lesson 10 compliance pass (done): Handouts and Keys for all eight engines now follow Rules 11, 13, 14 and 15 (shared state on the manager object, explicit loop/counter names, complete highlighted Level 1 code, Key code boxes for Levels 1-3). Rule 8 (slide speaker notes copied from the Teacher's Script, plus a new Activity Diagram slide) and Rule 9 (restart-flow Activity Diagram in all Handouts/Keys/Slides; GameMaker Visual and Unreal Blueprint node diagrams) are also done. Remaining: the GameMaker Visual R-key block is confirmed as 'If Key Pressed' (Letters > R), and the Unity Visual path is deferred (see above); Lessons 11-15 still need the same Rule 13/14 pass. *(Lesson 10 review)*
- ☐ Reflex habits: testing, documentation and refactoring as things students do without thinking of them as separate tasks (Robert's goal, raised at the Lesson 11 review after his own GameMaker project drifted from the Keys): see the separate document Reflex_Habits_Design_Notes.docx (this folder). It holds Robert's problem statement,

Claude's first-pass commentary (yes at beginner level if taught as three plain habits: try it, say it, tidy it, in the last five minutes of lab), a draft candidate Rule 16 (Done Means Tried, Named and Tidy), concrete mechanisms, options for fixing the instructor's own drift, a long spitballing list, risks, and the decisions needed. Nothing in it is adopted yet. *(raised by Robert at the Lesson 11 review; decision pending on candidate Rule 16)*

- ☐ Name the bug: "the Stalemate" (book edition, Robert's decision at the Lesson 11 review): refer to the win/lose gap bug by this name everywhere in the book and course materials from now on. Definition for the manuscript: a round that ends with no win, no loss and no restart, because escapes lower lives but never lower enemies_remaining. The name should appear the first time the bug is reported (Lesson 10's Level 3), be used in Lesson 11's wave discussion, and in any Semester 2 repeat of the bug-hunt exercise. The name also appears in Lesson 10 and Lesson 11 handouts and Teacher's Keys as of Lesson 11 (Lesson 10's own Level 3 text does not name it yet, only the Lesson 11 materials do; decide whether to add the name to Lesson 10's Level 3 in the book). *(decision at the Lesson 11 review; name adopted from Robert)*
- ☐ Lesson 11 compliance pass (done for active engines): Handouts and Keys for Scratch, Godot, GameMaker (script and Visual) and Unity were rewritten to match the Lesson 10 design: wave and total_waves live on the GameManager (Rule 13), the Lesson 10 spawn routine is reused as the 'spawn a wave' routine (spawn_enemies / SpawnEnemies, broadcast [spawn enemies]), complete highlighted Level 1 code (Rule 14), Key code for Levels 1 to 3 (Rule 14), Activity Diagram (new, diagram_activity_wave_flow) and a GameMaker Visual node diagram (Rule 9), game-state check for the new state, speaker notes copied verbatim from the Teacher's Script (Rule 8), Unity demo rewritten without the singleton (Rule 10). The Stalemate is deliberately left in for students to find and fix (Level 3), with a worked wave-aware fix in each Key. Paused engines (Unreal, 001 Game Creator, GDevelop, TIC-80, Unity Visual) were NOT rebuilt: their Lesson 11 files in the lesson folder are the old, stale versions (singleton, spawn_wave naming, no Rule 14 code) and must be rebuilt from the active-engine design during the book pass. *(Lesson 11 review, Rules 8, 9, 10, 11, 12, 13, 14, 15)*
- ☐ Unity static GameManager - concrete cost found in Lesson 11: because RegisterKill() is static, it cannot use the instance Spawner slot, so the wave request is passed through a static flag (nextWaveRequested) that GameManager's Update() acts on. It works and teaches 'static methods cannot reach instance members', but it is exactly the workaround the non-static redesign (see the static-members items above) would remove: with an instance GameManager, RegisterKill() would simply call spawner.SpawnEnemies(). Use this as the motivating example in the book's static-versus-instance discussion. *(found while rebuilding Lesson 11 Unity, Lesson 11 review)*
- ☐ Verify at the keyboard (Lesson 11, not tested by the author): (1) Godot: the next wave is requested with get_parent().call_deferred("spawn_enemies") because a wave clears inside a collision callback (suspected 'flushing queries' restriction on adding physics objects); confirm whether the deferral is actually required or merely harmless, and update the explanation accordingly. (2) GameMaker Visual: the Else block placement after a nested If Variable, and the Draw Value block's field labels, were written from the earlier Lesson 8 to 10 block names and have not been clicked through. (3) Scratch: confirm the last-hit clone never double-spawns a wave (it deletes itself before GameManager's broadcast arrives). *(Lesson 11 review, awaiting Robert's testing)*
- ☐ GameMaker Prefabs: GameMaker has a Prefab concept. Unity introduces Prefabs in the course, so decide where and how GameMaker's equivalent is introduced (likely alongside the lesson where Unity first uses Prefabs), verify what GameMaker Prefabs

actually are and how they differ, and update the cross-engine consistency reference and Data Dictionary accordingly. *(raised by Robert; research needed)*

- ☐ Resolve or explicitly caveat GameMaker's Lesson 13 explosion scale-origin risk before that lesson is written up. *(Task #243, still open)*
- ☐ Finish auditing Unreal Lessons 5, 6, 8, 9, 11, and 14 for the same missing "Split Struct Pin" step found (and fixed) in Lessons 2, 3, and 7. *(Task #254, partially resolved)*
- ☐ When describing Lesson 14 in the manuscript, explicitly call out that its single free-choice Level 3 (breaking the Sound/Visual/Structure template used every lesson since Lesson 2) is intentional scaffolding removal, not an inconsistency. *(found while building Cross_Engine_Consistency_Reference.docx)*
- ☐ Get Robert's decision on 001 Game Creator sprite import steps before that engine's chapter/section can be considered complete. *(Task #209, waiting on Robert)*
- ☐ Before claiming full 8-engine coverage anywhere in the book, finish QA-validating every engine in every mode (visual/script/block), not just the current focus subset (Scratch, Godot, Unity Scripting, GameMaker Scripting, GameMaker Visual, Unreal). *(Task #249)*
- ☐ Revisit lesson pacing across Lessons 3-15 once the full semester has actually run with students - note anywhere the real classroom pace diverged from the written plan before treating the lesson plans as the book's final word. *(Task #119)*
- ☐ Decide whether/how the course's companion digital materials (Handouts, Keys, GDDs) get referenced or reproduced alongside the book - affects how self-contained the book needs to be. *(Task #128, still open)*
- ☐ Check whether IGDA's Game Education SIG has released their updated Curriculum Framework yet, and reconcile the book's framework-alignment claims against it if so - the version available at project start (v3.2 beta, 2008) was already old, and an update was rumored to be in progress. *(see custom project instructions)*
- ☐ Periodically review and consolidate this document's Notes & Commentary section - merge duplicates, resolve contradictions between earlier and later notes, cut anything that turned out not to matter. Do NOT do this reflexively every time a note is added; batch it into occasional dedicated passes. *(standing item - see How To Use This Document)*
- ☐ Write a Lesson 4 Sidebar (or a Rule 6 Refinement and Style section, once that's a course fixture) on "who's in charge" - Bullet and Enemy each detecting their own hit and destroying only themselves, instead of one side reaching into the other. Ties to a live classroom discussion already introduced verbally but not yet formalized in writing. *(raised while checking Unity Lesson 4 sound handling, deliberately deferred rather than retrofitted)*
- ☐ Decide whether the 15 already-completed lessons get a retrospective retrofit pass for Standards Rule 6 (SuperStretch tie-in, Refinement and Style, visual-scripting node-reuse), or whether that content is written fresh for those lessons only when the book manuscript reaches them. See Notes & Commentary for the tradeoffs and a leaning recommendation - not yet decided. *(Standards Rule 6, added this session; scoping only, no retrofit started)*
- ☐ When rebuilding/auditing the Unreal Lessons, verify the Setup instructions actually include creating a no-Pawn Game Mode Blueprint (parent Game Mode Base, Default Pawn Class set to None in its Class Defaults) and setting it as the project's Default Game Mode - and check World Settings > Game Mode Override on each lesson's level too, since an override there takes priority over the project default. Confirmed missing/lost from Robert's own reference project this session: without it, the engine auto-spawns and possesses a stray built-in DefaultPawn at BeginPlay, which caused a hard-to-diagnose

symptom (a black sphere rising out of view, tied to Space being a legacy default "move up" binding on DefaultPawn colliding with Fire also being bound to Space) plus spurious collision-notice firings. *(confirmed via Robert's hands-on Unreal rebuild this session; see also the DefaultPawn note already under Notes & Commentary)*

- ☐ Reminder for Robert, not a course-content item: once Lesson 05 testing starts on his own project, go back and add Lesson 4's Color Vector Parameter tint / Lesson 5's striped-row idea into his own per-engine test projects - he skipped it during Lesson 4 testing as optional/untested, but confirmed (by reading the actual Lesson 4/5 Unreal content) that it's genuine forward-designed setup, not filler. *(personal test-project follow-up, not a document edit)*
- ☐ Reminder for Robert, not a course-content item: re-check his Scratch project's two enemy boundary values (the escape check at y position < -180 instead of the Key's -170, and the enemy spawn y of 130 instead of the Key's 150) by actually playtesting at full screen rather than only in the editor's small inset play window, since both were flagged as "observed placement" that may read differently at a larger render size. Worth noting for when he does: Scratch's own stage coordinate system (x: -240 to 240, y: -180 to 180) is fixed regardless of window size - only the rendered pixel scale changes, not the logical boundaries - so if -180 does cause a visible pop, it is equally true in both the small window and full screen; what's likely to change with screen size is how NOTICEABLE a small pop is, not whether the numbers themselves are correct. Worth keeping both possibilities open when he retests. *(personal test-project follow-up, not a document edit; Scratch coordinate-system clarification included for when Robert revisits it)*
- ☐ Retrofit past lessons' visual-scripting alternate-path Handouts and Teacher's Keys against Standards Rule 7 (name every block, category, and field - not just the shape of the logic). GameMaker's Lesson 5 GML Visual altPath was the first case caught and fixed, while testing his own reference project surfaced two real gaps: the setup step didn't distinguish deleting a room instance from deleting the object asset, and the altPath's note wrongly implied the Level 3 color-switch idea needed no changes from the code path when it actually needed its own block sequence. Audit every other lesson's GML Visual, Unity Visual Scripting, GDevelop, 001 Game Creator, and Unreal Blueprint alt-path content the same way - unlike Rule 6, Robert has directed this one to be retrofitted, not deferred to book-writing time. *(Standards Rule 7, added this session per Robert's explicit direction; Lesson 5 GameMaker, Lesson 8 (GameMaker/Unity altPath field/node explicitness), and Lesson 9 (Unity Visual Scripting altPath node-name explicitness) done, Lessons 2-4, 6-7, and 10-15 not yet audited)*
- ☐ Retrofit Lessons 1-4, 7, and 10-15's existing Slide Decks (.pptx) with Teacher's Script content in PowerPoint speaker notes, per Standards Rule 8. Lesson 5's deck already has this (each content slide's addNotes() carries that slide's matching Teacher's Script section verbatim, including bracketed stage directions) and is the reference example to follow; Lessons 6, 8, and 9 now also have full coverage (each lesson's six new notes calls follow the same slides-5/7/9/11/13/14 pattern Lesson 6 set, skipping title/overview/divider slides). Robert's classroom now runs Presenter View across two displays, so the notes pane doubles as his live script during lecture - this is a genuine workflow need, not just a nice-to-have, but Robert explicitly deferred the retrofit rather than asking for it done immediately. *(Standards Rule 8, added this session per Robert's explicit direction; Lessons 5, 6, 8, and 9 done, Lessons 1-4, 7, and 10-15 not yet started)*
- ☐ Retrofit Lessons 1-4, 6, and 10-15 against Standards Rule 9 (diagrams): add the generic Activity Diagram to every already-shipped Handout, Key, and Slide Deck; add engine-specific node diagrams to GameMaker/Unity/Unreal's Handouts (Level 1, unconditional) and Keys (Level 1 always, plus a combined Level 3 sequence diagram

where the visual path is complex enough to warrant one); and add diagrams to existing Sidebars where they'd genuinely clarify the text, per the same rule's Sidebars/Slides guidance. *(Standards Rule 9, added this session per Robert's explicit direction; Lessons 5, 7, 8, and 9 now have this treatment, Lessons 1-4, 6, and 10-15 not yet retrofitted)*

- ☐ Apply Standards Rule 9's diagram standard to Lessons 16-30 as each is authored, the same way it's now required for Lesson 5 onward - don't let the not-yet-built future lessons slip back to the pre-Rule-9 no-diagram default. *(Standards Rule 9, forward-application reminder)*
- ☐ Expand the Lessons 5-14 lecture-demo rotation (Standards Rule 10) from its current 5-choice pool (Scratch, GameMaker script, GameMaker visual, Unity C#, Godot) to evenly cover all 8 engines and all 3 coding styles, once students are actually using the currently-unused engines (GDevelop, TIC-80, 001 Game Creator, Unity Visual Scripting, Unreal Blueprints - none have active students right now). The original 30-Lesson Master Schedule already models what full-diversity balance looks like and can be adapted rather than redesigned from scratch. Until then, the restricted 5-choice rotation is intentional, not an oversight. *(Standards Rule 10, added this session per Robert's explicit direction to narrow the pool for now)*
- ☐ Retrofit past lessons' code and visual-scripting content against Standards Rule 11 (meaningful variable names over generic i/j/k counters), across all engines and coding styles. Lesson 5's audit is now complete across all 8 engines: GameMaker (Code and Visual paths, i -> enemy_count, inst -> new_enemy), Godot (Code path, i -> enemy_count, e -> new_enemy, plus a ternary -> if/else cleanup), Unity (Code path, i -> enemyIndex, e -> newEnemy, ternary -> if/else), and TIC-80 (Lua, i -> enemy_count in the spawn loop only - the separate bullets-iteration and enemies-array-iteration loops use i/j for a different concept and were deliberately left untouched) all needed and got a rename. Unreal's C++ alt-path (i -> EnemyCount) also needed one; its Blueprint path needs none (the bare "Index" it uses is the ForLoop node's own fixed pin name, not a variable this course chose - same as Unity Visual Scripting's Index pin). GDevelop and 001 Game Creator were both audited and found to need no change, for two different reasons: GDevelop's self-declared EnemyIndex scene variable was never generic to begin with; 001 Game Creator has no loop or variable at all for this concept - Level 1 is pure manual copy/paste placement in the Scene editor, and Level 3's row variety is manual per-instance Tint editing, so there's nothing to rename. Lesson 5 is done; every other lesson (1-4, 6-15) has not yet been audited for the same generic-counter pattern. Robert has adopted a standing workflow for this going forward: rather than a dedicated retrofit sweep, run a quick Rule 11 naming pass on an engine's Lesson 5+ content the next time he personally starts testing that lesson/engine combination, opportunistically, as he works through the course. *(Standards Rule 11, added this session per Robert's explicit direction; Lesson 5 fully audited across all 8 engines (GameMaker/Godot/Unity/Unreal C++ renamed, Unreal Blueprint/GDevelop/001GC confirmed no change needed, TIC-80 renamed), opportunistic per-lesson-as-tested workflow adopted for Lessons 1-4/6-15 in place of a dedicated retrofit sweep)*
- ☐ Pick a specific Semester 2 lesson to formally introduce the ternary operator, once Semester 2's actual lesson content is being built - Robert's own suggestion, prompted by pulling the ternary out of Lesson 5's GameMaker code for being too advanced this early. Not yet scheduled to a lesson number; needs to land somewhere a student is already comfortable with plain if/else and is extending an existing codebase, not meeting conditionals for the first time. *(raised alongside the Lesson 5 ternary removal; see the matching Notes & Commentary entry)*
- ☐ Rename Scratch's Lesson 5 loop variable from enemy_number to enemy_count, to match GameMaker and Godot. Robert's decision: enemy_count wins the naming

question outright (it already had two engines using it before this was even raised as a question), so Scratch - the outlier, and the only engine using enemy_number - gets reconciled to match rather than the other two being changed. Not yet done as of this entry. *(Robert's explicit decision, given while Unity's Rule 11 retrofit was underway; supersedes the earlier not-yet-decided framing of this same question)*

- ☐ Run a dedicated, coordinated pass reconciling BOTH variable/function naming AND code/graph structure across all 8 engines and all 3 coding styles, per new Standards Rule 12 - broader in scope than the existing opportunistic, per-lesson-as-tested Rule 11 naming workflow (above), which only catches naming, one engine at a time, as Robert happens to test it. This pass should also resolve every deliberately-deferred word-choice and structural conflict surfaced along the way rather than fixing any one of them in isolation. Known cases so far, all in Lesson 5's spawn-loop concept alone: Unity's enemyCount (existing total-to-spawn field) vs. enemyIndex (the Rule-11-compliant but non-matching loop counter name used to avoid shadowing it); Scratch's enemy_number vs. the enemy_count used everywhere else (Robert's decision: enemy_count wins, tracked as its own ToDo above); GDevelop's EnemyIndex, meaningful but a different word choice ("index") than the "count" used by GameMaker/Godot/Scratch-once-renamed; and Unreal Blueprint's spawned-actor reference, which is never promoted to a named variable at all (wired directly from Spawn Actor from Class's Return Value), unlike every other engine's named new_enemy/newEnemy - a structural gap, not just a naming one. Also now known: which object owns bullet/enemy collision detection and response is opposite between GameMaker and Godot - GameMaker's obj_bullet has the Collision event and destroys the enemy (and itself) from the bullet's side, while Godot's enemy.tscn owns an Area2D Hitbox (area_entered signal) and destroys the bullet (and itself) from the enemy's side. Neither is wrong on its own, but nobody ever decided which object should own this across engines - it just landed differently as each lesson was authored. Robert's own refactoring preference (each object cleans up after itself / is self-reporting, surfaced while reviewing his GameMaker and Godot projects) points toward a third option worth considering in the eventual pass - the thing that dies announces its own death - rather than simply picking one engine's existing side to standardize on. Robert's explicit decision when this was raised: track it only, do not unwind it for current students now - that reconciliation is real but belongs entirely to this eventual coordinated pass. Also now known, found while Robert reconciled his own Godot project: spawn_enemies() has the same NAME in GameMaker and Godot but a different OWNER - GameMaker's is scr_spawn_enemies, a free-standing global Script resource belonging to no object, callable directly by anything (the bullet, start_game(), wave_cleared()); Godot's is a method on main.gd, owned specifically by the Main scene node, with GameManager having to reach it via get_parent().spawn_enemies(). This is tangled up with a second asymmetry that cuts both ways: Godot has no parallel to GameMaker's obj_spawner, but GameMaker equally has no parallel to Godot's main.gd - GameMaker has no scripted scene-root object at all (the Room is unscripted; obj_gamemanager is just a peer object in it), so main.gd isn't really standing in for obj_spawner, it's filling a role (scene-root ownership of the spawn routine) that doesn't exist as a GameMaker concept at all. Note this partly resolves on the GameMaker side once the planned obj_spawner retirement (see the six-candidate-refactorings ToDo item) happens - at that point GameMaker's spawn-related logic becomes entirely global-Script-resource-based with nothing object-owned, while Godot's would still be split across two node scripts - a real design-philosophy fork (centralize all shared logic on one manager, vs. let each engine's own scene-ownership idiom decide where code lives) worth its own discussion in the eventual Rule 12 write-up, separate from simple naming mismatches. Robert has also asked, once this reconciliation pass happens, to

extend the same alignment effort to Scratch's objects/events/broadcasts - matching Scratch's existing GameManager sprite and broadcast-message names to the equivalent function/event names used in the other 7 engines as closely as Scratch's own block-based model allows, acknowledging this may land more as explicit cross-references in the Data Dictionary (broadcast message X = function Y in engines A/B/C) than as literal renaming, since Scratch's own mechanisms won't map one-to-one. Updated again to record that Robert's own GameMaker/Godot ownership-reassignment drift (see the dedicated ToDo item above) is the same underlying pattern as this collision-ownership asymmetry for those two engines - Robert's own projects actively explored both directions (bullet-owns vs. enemy-owns) before this Rule 12 question was ever raised formally. Updated again to add a FOURTH engine data point, found while Robert reconciled his own Unity project: his Bullet.cs's Start() plays its own fire/shoot sound via a private AudioSource field - a deliberate ownership choice, confirmed explicitly by Robert and kept as-is (not reconciled to canonical's Player-owns-the-shot-sound placement) specifically so it can be documented later for the "Doing It the Doers of Stuff Way" chapter - and his Enemy.cs had taken over OnTriggerEnter2D collision detection/response (RegisterKill(), hit sound, destroying both objects) that canonical keeps exclusively on Bullet.cs, the same bullet-vs-enemy ownership swap already tracked for GameMaker and Godot above. Unlike the fire-sound placement, Robert asked for the collision-ownership split to be reconciled back to canonical for the current course version, which was done (collision detection, the hit AudioClip, and RegisterKill() moved back onto Bullet.cs; Enemy.cs now only moves, checks the off-screen threshold, and calls RegisterEscape()). CORRECTION, same session: Scratch is NOT an example of Robert's drift the way GameMaker and Godot are - Scratch's own shipped Key deliberately switches from bullet-owns (Lessons 3-4) to enemy-owns (Lesson 5 onward) ON PURPOSE, for a genuine Scratch-specific technical reason stated directly in the Lesson 5 Key: Scratch clones have no unique ID, so only the Enemy clone being touched can identify and delete itself - the Bullet side has no way to know which specific Enemy clone it hit. This makes Scratch a THIRD real data point for this list (enemy-owns, like Godot, but for a completely different and well-documented reason), and it also means Scratch's own Lesson 3-4 vs. Lesson 5+ materials are themselves internally inconsistent about which object the Handouts/Keys show owning collision - worth flagging as its own small audit item (do Lessons 3-4's materials still read as if that design persists forward, or do they already set reader expectations correctly for the Lesson 5 change) during the eventual Rule 12 pass. Robert's stated reasoning: the goal throughout is enabling students on different engines to help each other, which is worth pursuing mindfully even where a perfect one-to-one mapping isn't possible. This pass should also settle the open PascalCase-vs-camelCase-vs-snake_case question once and for all (see the separate ToDo item on that question) so the coordinated naming pass has one settled convention to apply rather than re-litigating case style engine by engine. Updated again to record a new, confirmed case found during Lesson 14's compliance pass: TIC-80 genuinely and correctly calls its spawn routine spawn_wave() - NOT spawn_enemies() like Godot, GameMaker, and Scratch, and NOT SpawnEnemies() like Unity. This was discovered specifically because fixing a spawn_wave()-vs-spawn_enemies() drift bug that affected four OTHER engines required definitively confirming TIC-80's own correct name before leaving it alone, which surfaced a real, previously undocumented two-way split (spawn_wave vs. spawn_enemies/SpawnEnemies) worth folding into the eventual coordinated pass alongside the existing cases. *(Standards Rule 12, added this session per Robert's explicit direction; a fundamental precept (common ground across engines so students can help each other regardless of engine/style), not yet started; growing list of known reconciliation cases tracked here as they're found)*

- ☐ Decide the course's cross-engine variable/function naming CASE convention (PascalCase vs. camelCase vs. snake_case vs. some other scheme) as part of the Standards Rule 12 reconciliation pass (above) - a distinct question from which underlying WORD an engine uses (e.g., enemy_count vs. enemy_number), which is about the letter-casing/formatting style layered on top of whichever word is chosen. Robert's stated inclination: favor cross-engine consistency in the actual case/format used, even in engines where a different case convention is the idiomatic local norm (e.g., Unreal/C++ typically PascalCase, C-family languages typically camelCase, GML/GDScript typically snake_case) - and where the chosen course-wide convention deviates from an engine's own idiom, explain that deviation explicitly in a Sidebar rather than silently picking the engine-idiomatic case and letting the inconsistency stand unexplained. Not yet decided which specific convention wins; this entry only records the framing and Robert's leaning, not a final choice. *(raised alongside the Data Dictionary discussion; Robert's explicit inclination given, final convention choice deferred to the Rule 12 pass)*
- ☐ TIC-80's bare loop variables j and e (the enemy-array iteration pattern, e.g. "for j=1,#enemies do" / "local e=enemies[j]") were renamed to enemy_index/enemy for Lessons 7 and 8 as part of each lesson's own compliance pass. The identical unrenamed pattern was confirmed, by direct grep, still present in Lessons 5 and 6's build scripts. Robert's lesson-scoped authorizations for those passes didn't extend to Lessons 5-6, so this was deliberately left as a follow-up rather than done opportunistically alongside them. Fold into either a dedicated Rule 11 TIC-80 sweep or the broader Rule 12 reconciliation pass (above), whichever comes first. *(found during Lesson 7's Rule 13/diagram compliance pass, then confirmed already fixed for Lesson 8 during that lesson's own pass; Lessons 5-6 still open)*
- ☐ Decide, for the book, whether Unity's early lessons should introduce [SerializeField] private fields instead of (or alongside) plain public fields for Inspector-exposed values. Raised while Robert reconciled his own Unity project (EnemySpawner.cs uses [SerializeField] private enemyPrefab/enemyCount where canonical uses public). Robert's explicit framing: he changed it in his own project because he knows the distinction, but deliberately did NOT ask for the canonical course materials to be changed, because he isn't convinced the encapsulation conversation is worth it for a beginner class - it may belong in the "Doing It the Doers of Stuff Way" refactoring chapter instead, or nowhere at this course's level. His current leaning: keep canonical on plain public fields for simplicity, but he wants this decided intentionally, as its own discussion, not defaulted into either way. Not yet decided. *(raised while reconciling Robert's EnemySpawner.cs; Robert's own project differs from canonical on this point by his own deliberate, informed choice - not drift to fix)*
- ☐ Add gameActive = true; to Unity GameManager.cs's Start() in the shipped Lessons 10-12 Handouts/Keys (and anywhere else Start() resets score/lives/wave/hasWon/hasLost), alongside the existing resets already there. Confirmed, reproducible bug, found via Robert's own project: gameActive is currently only ever reset inside ResetGame() (the R-key restart), never in Start(). If a student's Unity Editor has "When entering Play Mode" set to "Reload Scene only" (Edit > Project Settings > Editor > Enter Play Mode Settings - a single dropdown in current Unity versions, not the two checkboxes older instructions might describe) rather than the default full reload, the scene's GameObjects are destroyed and recreated fresh on every Play session (so Start() does run), but static fields like gameActive are NOT reset (no domain reload), so they keep whatever value the PREVIOUS Play session left them at. If that previous session ended in an actual win or loss, gameActive is left false, and the next Play session's enemies spawn via Start() (unaffected) but never move, since Enemy.cs's own Update() starts with if (!GameManager.gameActive) return; - a silent,

total freeze, not a slowdown. Worth a dedicated troubleshooting-sidebar entry too: this is a legitimate, easy-to-enable Editor setting (students speeding up their own iteration time), not a mistake, so the gotcha is worth explaining on its own terms rather than just quietly patched away. *(confirmed via Robert's own project, diagnosed collaboratively over several messages; not yet fixed in shipped materials)*

- ☐ Reminder for Robert, not a course-content item: re-drag the Inspector field references (spriteDefault/Left/Right/Up/Down, sr, walkingBeep, xBoundary, yBoundary) onto the reconciled PlayerMovement.cs and playtest - the restructured single-moving-check version (fixing the diagonal-movement blink-rate bug and the missing walkingBeep.Stop()) has not yet been verified at the keyboard. *(personal test-project follow-up, not a document edit; last file in the Unity reconciliation sequence)*
- ☐ Fix the shipped Lesson 4 Unity Handout/Key for the book: AudioSource.PlayClipAtPoint(hitSound, transform.position) should be AudioSource.PlayClipAtPoint(hitSound, Camera.main.transform.position). This is a genuine, confirmed bug (PlayClipAtPoint() always creates a 3D-blend temporary audio source with real distance rolloff, so a sound played at the enemy's own position is quietly, almost inaudibly wrong, unlike every other PlayClipAtPoint call in the course, all of which already use Camera.main.transform.position) - found while Robert debugged his own quiet hit sound. Robert's explicit decision: do NOT fix the live Lesson 4 materials now, mark for book correction only. Lesson 13 forward was checked and fixed separately (see the matching Notes & Commentary entry) since it's live, current content. *(confirmed bug, found via Robert's own project; Lesson 4 itself deliberately left unfixed per Robert's instruction, Lesson 13 already fixed)*
- ☐ Write a Unity troubleshooting-sidebar entry on AudioSource.PlayClipAtPoint()'s quiet-sound trap: it always spawns its temporary audio source with 3D spatial blend and real distance rolloff, so a sound played anywhere other than Camera.main.transform.position (or very close to the camera) can end up nearly inaudible - and the optional volumeScale argument can't fix it, since it's a 0-1 percentage of the clip's own volume, not a boost, and can never compensate for rolloff. The practical fix used throughout this course is simply to always call PlayClipAtPoint at Camera.main.transform.position, the same pattern GameManager's round-ending sounds already used from the start. Good candidate for the same category of "common pitfalls" sidebar content already tracked for the color-multiply-by-white bug. *(raised while fixing Robert's quiet Bullet.cs hit sound and the matching Lesson 4/13 course bug)*
- ☐ Standards Rule 13 (dedicated manager object for shared/global state) has only been retrofitted into Lessons 6, 7, 8, and 9 so far (Scratch, Godot, GameMaker, and Unreal gained a GameManager-equivalent object; Unity already had one, which is what prompted the rule; Lesson 8 additionally retrofitted the lives/has_lost lose-condition state onto the same object, and Lesson 9 retrofitted score the same way across GameMaker, Scratch, Godot, and Unreal). Lessons 10-15 haven't been audited yet for the same pattern - any lesson whose Handouts/Keys have a spawner, player, or other object directly owning a shared counter (a win/lose flag, a score, a lives count) rather than routing it through a dedicated manager object is a Rule 13 candidate. Audit opportunistically, the same adopted workflow as Rule 11's naming retrofit, rather than as a dedicated sweep, unless Robert says otherwise. *(Standards Rule 13, added this session; Lessons 6-9 done, Lessons 10-15 not yet audited)*
- ☐ Godot-specific (no real parallel in the other 7 engines, per Robert): decide, for the book, which way of connecting a signal is better/more idiomatic/more consistent/easier to track and debug - writing the connection explicitly in code (e.g. $Hitbox.area_entered.connect(_on_hitbox_area_entered) inside _ready(), the way the

shipped Keys show it) versus wiring it point-and-click through the Editor's Signals tab (which Robert's own project uses instead - the connection then lives in the scene file's metadata, not as a visible line in the script at all). Raised while Robert reconciled his own Godot project's enemy.gd against the Lesson 6 Key and noticed his version had no .connect() line because he'd wired it through the Signals tab instead. Explicitly deferred - track only for now, do not change the live course's current approach (code-based .connect(), as already shipped) while reconciling Robert's own project. *(raised by Robert while reconciling his Godot project; Godot-only, no cross-engine parallel; deferred to the book pass)*

- ☐ For the book manuscript, move the GameManager mini-lecture ("Before We Start: Meet the GameManager" / "Why a Dedicated Manager Object?") from where it's live-taught to where it actually belongs conceptually: Lecture 06, not Lecture 07. The two slides live in Lesson 7's deck right now purely for production-timeline reasons - the retrofit was designed and caught after Lesson 6 already shipped, so it had to land as a lab-time catch-up notice in the next lesson taught, not the lesson the object was actually introduced in. The book isn't bound by that same live-classroom constraint, so this content should be written into Lecture 06's chapter instead, presented as part of introducing the win condition and shared state in the first place, with Lesson 7's chapter only needing a brief backward reference rather than the full explanation. Keep the two slides in Lesson 7's live deck as-is; this is a book-only reorganization, not a live-course change. *(raised alongside adding the GameManager mini-lecture slides to Lesson 7's deck; book-only reorganization, live deck unaffected)*
- ☐ New companion book/class concept, spitballed, not scoped or scheduled: "Debugging 101" - a course/book built around a pre-loaded code repository seeded with a variety of deliberately planted bugs, modeled directly on real categories this very project has hit: flat-out broken code that doesn't compile/run, failed edge cases, a new feature that breaks an existing one, a mismatch between the design doc and the actual code, a broken refactoring, and a misinterpreted feature (built to spec as written, but not to what was actually meant). Natural companion idea: "Testing 101" - the inverse situation, an existing codebase plus a design doc but NO test cases, where the student's job is to build the test suite from scratch. A possible third, combined course: "Testing/Debugging 201", merging both - use the act of writing test cases as the mechanism that both finds the bugs AND proves each fix actually worked (red test before the fix, green test after), rather than teaching testing and debugging as separate skills. All three are genuinely downstream of THIS project specifically: the bug categories map directly onto real bugs found while building this course (see the Notes & Commentary entries on the spawn-loop self-collision bug, the GameMaker Visual targeting saga, the dropped Level 3 sounds, the WaveAnnouncementText/WaveText naming mismatch, and the Stalemate itself), which is exactly the kind of real-world-grounded material this idea would need. Not scoped, not scheduled, no engine/language/audience decided yet - purely a note-it-before-it's-lost entry; see the matching Notes & Commentary entry for the fuller framing. *(Robert's idea, spitballed; companion concept to the main Video Game Creation course, not yet scoped)*
- ☐ GameMaker GML Visual audit: confirm whether any existing Visual-path content assumes an "Else If" node/block exists. It appears GML Visual has no such block - a three-or-more-way branch (if / else-if / else-if / else) has to be built by nesting a second If Variable (or equivalent) block inside the first one's Else slot, one level deeper each time, rather than chaining same-level blocks the way code's else if does. Any Handout, Key, or diagram instructing a student to "add an Else If block" in GML Visual is describing a block that may not exist and needs rewriting as nested If/Else instead; any node diagram showing a flat else-if chain for GameMaker Visual needs the same correction, drawn as actual nesting. Not yet audited against the shipped lessons - verify at the keyboard

which lessons' GML Visual content has a 3-plus-way branch (Lessons 8-9's lives-vs-enemies-remaining check and Lesson 11's wave-cleared check are the most likely candidates) and confirm whether they already nest correctly or need fixing. *(raised by Robert; verify at the keyboard before concluding which lessons are affected)*

- ☐ Extend Standards Rule 12 (cross-engine consistency) to cover pop-up/overlay text presentation, not just variable and function naming: wave announcements, win/lose banners, and other HUD or momentary-message text should look as similar as reasonably possible from engine to engine - specifically including things like centering on screen, vertical placement, and text size, which the course has so far let drift silently (Robert's own words: "I've silently deviated on a few things like text placement, size, and such"). Proposed shape of the fix: define one simple, explicit set of reasonable defaults (e.g., centered horizontally, a consistent vertical band for a given message role, a consistent relative size for "HUD counter" text vs. "big announcement" text) and apply it as the baseline in every engine's Handout/Key, while still leaving room for a student to deviate on these small things if they want to. Robert's explicit design goal for how this should read in the lesson materials: it should take very little callout in the lesson plans themselves - the consistency should mostly just BE there, quietly, rather than being taught as its own topic - while still being fairly obvious to a student who looks at the actual steps what the convention is and why those particular values were chosen. A secondary benefit Robert flagged on purpose: nailing down one specific way to do a small, otherwise-you'd-have-to-look-it-up task (centering text on screen in each engine, for instance) functions as a tiny, free lesson in that engine's own text/UI API that the course doesn't have to spend any lecture time on, but that saves a student from going and searching for it themselves. Needs: an actual audit of every existing pop-up/HUD text element across the shipped lessons to find where defaults have already silently diverged, a decided set of default values per message role, and a lightweight way to state the convention in each engine's Setup/instructions without it reading as a big teaching moment. *(raised by Robert; extends Rule 12's existing scope from naming/structure to visual presentation of HUD/overlay text; not yet audited or decided)*
- ☐ Book retrofit (Robert's decision, raised while reviewing Lesson 12): remove ALL sound assets and sound programming currently scattered as optional Level 3 ideas across the earlier lessons (the movement blip in Lesson 2, the shoot sound in Lesson 3, the hit/explosion sound in Lesson 4, the victory sound in Lesson 6, the life-lost/game-over sounds in Lesson 8), and reserve sound entirely for a dedicated Lesson 12 that teaches it properly as its own concept, not as a scattered series of optional add-ons. Robert may still choose to keep some of these as stretch/Level 3 goals in individual earlier lessons, but the explicit design intent either way is to make a conscious effort to "pull it all together" in Lesson 12 rather than let sound arrive piecemeal. This is a book-only retrofit - the live course's Lessons 2-11 already shipped with the scattered-optional-sound design and are not being changed retroactively; Lesson 12 itself was already corrected this session to assume every one of those sounds might be either freshly built or already present (see the Lesson 12 compliance pass note above), so the live course's actual Lesson 12 content does not need to change for this decision - only the earlier lessons' content would, in the book version. See the matching Notes & Commentary entry for Robert's full reasoning and its connection to the Level 1/2/3 framework question also raised in this same message. *(Robert's decision, raised reviewing Lesson 12; book retrofit only, live course Lessons 2-11 unaffected)*
- ☐ Clarify and document the intended PURPOSE distinction between Level 1 and Level 2 across the whole course (Robert's framing, raised alongside the sound-consolidation decision above): Level 1 (Minimal/MVP) exists specifically so a classroom-paced student

has a safety net - something they can reach quickly enough to not fall so far behind the live class cadence that they can no longer reasonably absorb the next lesson. Level 2 (Complete) is the actual PREFERRED accomplishment level, especially for self-paced/individual study, because it represents a more realistic, complete feature set and deliberately mixes "spoon-fed" instruction with "think on your own" challenge in a way Level 1 alone does not. This reframes Level 1 as a classroom-pacing safety valve rather than the course's real target, with Level 2 being the actual goal the course is designed around - worth stating explicitly in the book's framework chapter (see the existing "Level 1/2/3 framework as the book's spine" Notes & Commentary entry) rather than leaving the Level 1 vs. Level 2 relationship implicit. A direct consequence for the sound-consolidation decision above: Robert is fairly sure Level 2 should NOT include sound effects once sound is consolidated into Lesson 12, since Level 2 is the realistic/preferred target students should reach across the earlier lessons, and sound doesn't belong there until its own dedicated lesson - not yet finalized, but stated as a strong leaning. *(Robert's framing, raised alongside the sound-consolidation decision; affects how Level 1/2/3 goals are worded across the book, not yet drafted)*

- ☐ Six candidate refactorings for the "Doing It the Doers of Stuff Way" final chapter (Robert, spotted while reconciling his own GameMaker Script project back into compliance with the shipped Lesson 12 code): (1) enemies_remaining should be incremented directly inside each enemy's own Create event rather than set via a fixed count inside the spawn loop that creates them - lets each enemy be self-reporting rather than relying on the spawner to know the total. (2) The number of enemies to spawn (currently a "mystery number" - 5 - hardcoded inside the spawn loop) should instead be a named variable (something like enemies_per_wave), so the value's meaning is visible at the point it's used, not just inferable from context. (3) enemies_remaining checks/decrements are currently scattered across multiple locations (obj_bullet's Collision event, obj_enemy's Step event, both now also routed through wave_cleared()) - Robert thinks most or all of this logic should consolidate onto a single, dedicated location: obj_enemy's own Destroy event, so that however an enemy goes away (hit or escape), the SAME one place is responsible for updating the count, rather than each removal path updating it independently. (4) The player-lives check (currently __dnd_lives, read/written from obj_enemy's Step event on an escape) arguably doesn't belong on obj_enemy at all - Robert leans toward it living on either obj_player or obj_gamemanager instead, since lives is fundamentally about the player/game state, not something the enemy itself should be reaching in to modify. (5) obj_player's Step event has accumulated enough logic over the semester (movement, firing, and whatever else has landed there) that moving movement specifically into its own separate script would reduce clutter and give Step a single, easier-to-scan responsibility. (6) obj_spawner's sole remaining job as of Lesson 10 is a single-line Create event (start_game();) - eventually that one call could move onto obj_gamemanager's own Create event instead, retiring obj_spawner as a distinct object entirely, since it no longer does anything a GameManager couldn't just as easily do for itself. Robert specifically noted that obj_spawner's own HISTORY across the course - starting as the object that owns the entire spawn loop (Lesson 5), shrinking to a single delegated function call once start_game() existed as its own Script resource (Lesson 10), and potentially being retired altogether as a final step - is itself a good, concrete, already-real example of incremental refactoring for the final chapter to walk through, not just the end-state change. None of these six are being implemented now - explicitly captured as raw material for the eventual dedicated refactoring-walkthrough chapter, where the WHY behind each one (self-reporting objects, named vs. magic numbers, single point of responsibility for a shared counter, state living on the object it actually belongs to, extracting a script to

reduce event clutter, and retiring an object once its responsibilities have all been absorbed elsewhere) can get the full discussion Robert wants that chapter to have. Confirmed cross-engine while reconciling his Scratch project: Robert had already independently built candidate (1) (self-reporting enemy count) there too - each Enemy clone broadcast its own creation to GameManager, which incremented enemies_remaining per-clone rather than a flat set-to-5 - and said he "probably did this in every engine." This is being reconciled back to the canonical flat-set design in the live course materials (not kept), per Robert's explicit decision, but confirms the idea wasn't a one-off thought - he built toward it independently and repeatedly before it was ever written down here, which is itself worth mentioning when the final chapter discusses candidate (1). *(Robert's own refactoring observations, raised while reconciling his GameMaker project; explicitly for the final refactoring chapter, not implemented now; item 6 doubles as a real example of incremental (not one-shot) refactoring; candidate (1) independently confirmed built (then reconciled away) in Scratch too)*

- ☐ GameMaker Lesson 12 sound calls given at the Level 1 architecture (inline win/wave-advance if/else in obj_bullet, direct spawn_enemies() call) may need reconciling against a project that's already past Level 1 - specifically, one that has the Lesson 10/11 Level 3 "Stalemate" fix in place (a shared wave_cleared() function, called from both obj_bullet's hit path and obj_enemy's escape path, so an escape can also clear a wave). If wave_cleared() already exists in a given project, Lesson 12's sfx_wave call belongs ONCE, inside wave_cleared()'s else branch, not duplicated/left inline in obj_bullet's Collision event. Raised while Robert was reconciling his own GameMaker Script project (which has deviated furthest from the shipped course materials of any engine) against the corrected Lesson 12 Key. Explicit decision: do NOT retrofit this now, in either the live course's shipped Lesson 12 materials or Robert's own test project. Leave it be, and handle it one of two ways later - either as an explicit refactor if a later lesson's own progression genuinely requires the wave_cleared() consolidation to exist by that point, or fold it into the planned "Doing It the Doers of Stuff Way" final refactoring chapter as a worked example (a shared-function extraction prompted by a second call site appearing, which is a very natural refactoring story to walk through). *(raised while Robert reconciled his own drifted GameMaker project against the Lesson 12 Key; explicitly not retrofitted now)*
- ☐ NEW Lesson 15 deliverable, fully scoped and decided (Robert), but explicitly DEFERRED - do not start building yet: a "Complete Codebase" document per engine, handed to students at Lesson 15 so they can check their own work over the semester break and come back ready for Semester 2. Scope, as decided: (1) Engines - the active 5 (Scratch, Godot, Unity C#, GameMaker Code, and GameMaker Visual), matching the existing active-engine-scope decision; the 4 paused engines (GDevelop, TIC-80, 001 Game Creator, Unreal) are explicitly out of scope for this deliverable. (2) Depth - NOT just the reliable Level 1/2 baseline; Level 3 "complete code" should be included wherever it exists, which means first BACKFILLING the known gap where several lessons' Keys only describe Level 3 in talking-point prose rather than full code (see the Rule 14 retrofit ToDo item), rather than skipping those lessons' Level 3 content. Robert's explicit reasoning for choosing the harder option: doing that backfill work now feeds forward into the book version too, so it isn't wasted effort scoped only to this one deliverable. (3) GameMaker Visual - represented as full node-flow diagrams per object, not prose description, extending the existing wireframe-diagram pilot (Lessons 5/7/8/9/11) to cover every GameMaker Visual object/script in the finished project completely, end to end. CRITICALLY: Robert explicitly said NOT to start this now - wait until the remaining lessons (through at least Lesson 15) have been reviewed first, because building this deliverable before that review is likely wasted effort (the reviewed lessons may still change, which would mean redoing the Complete Codebase work).

Treat this as a planned, fully-scoped future task, not a current one - revisit once the remaining-lessons review is done. *(Robert's decision, fully scoped; explicitly deferred until the remaining lessons are reviewed - do not start early)*

- ☐ GameMaker naming inconsistency found and fixed: the drag-and-drop sound-playing block is called "Play Audio" (confirmed by Robert directly against his own installed GameMaker version). Several shipped Lessons (8, 10, 11, and the newly-added Lesson 12 content) had instead called it "Play Sound" or "Audio Play Sound" - genuine naming drift, not a deliberate choice. Fixed across build_lesson08/10/11/12_handouts_keys_checklist.js, build_lesson12_plan_teacher_script.js, and build_lesson12_slides_deck.js; rebuilt, PDF-verified (grep confirmed "Play Audio" present, no stale text remaining), and shipped to the Lesson 8/10/11/12 folders. Lesson 13's similar-looking "Play Sound"/"Play Sound 2D" references were checked and confirmed to belong to 001 Game Creator and Unreal Blueprint respectively - different engines' real terminology, correctly left untouched. *(Robert confirmed the correct block name at the keyboard; fix completed and shipped this session)*
- ☐ Book-only final chapter, decided (Robert): "Doing It the Doers of Stuff Way" - a dedicated, one-time refactoring walkthrough, taking the capstone's finished Semester 1 project and refactoring it end to end with full discussion of why and how, introducing Robert's own particular coding style and opinions explicitly as HIS style, not as a new ongoing requirement. This explicitly REPLACES an earlier, now-abandoned idea of a standing "Level 4" (refactoring + personal style, applied lesson by lesson alongside Levels 1-3 the same way Level 3/stretch goals and Sidebars already are). Robert's stated reason for rejecting the ongoing-Level-4 approach: it would force a choice between two bad options every single lesson going forward - either make refactoring-to-Robert's-style a mandatory part of every lesson (which he does not want), or maintain two parallel code paths per lesson (the plain Level 3 solution AND a separately-maintained "Refactored Level 3" solution) - and this project has already directly experienced how costly maintaining two divergent code paths for the same lesson can be (see the Script/Visual parity cleanup ToDo item and GameMaker's Code-vs-Visual drift problem elsewhere in this document) - a second one, chasing a moving personal-style target lesson by lesson, is explicitly something he does not want repeated. Instead, this becomes a single dedicated final chapter/lecture, intended to fall OUTSIDE the 30-lesson cadence entirely - delivered as a final micro-lecture, something students take home at the end of the course to study and experiment with on their own, explicitly framed as an invitation for students to develop their OWN opinions, form, and style rather than simply inheriting Robert's. Not yet scoped: which specific refactorings to walk through, how long the micro-lecture runs, or whether it's written as a live-delivered lecture, a stand-alone reading, or both. *(Robert's decision, supersedes the earlier unscoped Level 4/refactoring idea; book-only, falls outside the 30-lesson structure)*
- ☐ Robert's own test-project reconciliation (GameMaker, Godot, and now Scratch) surfaced a real pattern in GameMaker and Godot, but a CORRECTED finding for Scratch specifically: Robert repeatedly moved shared logic (bullet/enemy collision detection/response, the shoot sound's trigger point) to whichever object he personally felt should own it while exploring his own eventual refactoring preferences - confirmed genuine, self-introduced drift in GameMaker and Godot (both already logged above). For Scratch, Claude initially (wrongly) told Robert his Enemy-owns-collision code was the same kind of drift and should be reconciled back to a Bullet-owns-collision design - this was an error, caught and corrected in the same session: Scratch's OWN canonical Key design switches hit-detection from the Bullet (Lesson 3-4) to the Enemy (Lesson 5 onward, and every lesson through 12) on purpose, documented directly in the Lesson 5

Key: "Scratch clones have no unique ID, so a Bullet clone can't identify exactly which Enemy clone it's touching from the Bullet's side - that's why the check moved to each Enemy clone's own script." So Robert's Scratch Bullet's lack of a touching-check was already correct and needs no change - only the shoot-sound placement (Bullet's own clone script vs. the Key's Player-side placement) remains an open, genuine style question for Scratch, not a correctness bug. Net effect: the cross-engine ownership-reassignment pattern below is real for GameMaker and Godot, but Scratch's Enemy-owns-collision is the engine's actual canonical design, not Robert's drift - worth keeping both facts straight for the eventual Rule 12 pass and the refactoring chapter. Separately, Robert's Scratch Bullet also spawns offset 100 pixels from the Player's exact position (y + 100) rather than exactly on it, done consistently the same way across his other engines' projects too - a minor, deliberate, and apparently cross-engine-consistent deviation from the Keys' literal spawn position, not a mistake. Robert wants this one specifically noted and then removed when the book's canonical code is written, with the idea saved for the refactoring chapter rather than silently folded into the canonical design. *(Robert's explicit decision re: GameMaker/Godot ownership drift; Scratch correction found and caught within the same session after an initial wrong read by Claude)*

- ☐ Audit every other Scratch lesson for "set [color] effect" used as if it were a white/brightness blend (a tint), not just a hue rotation. Lesson 13's own shipped flash code made exactly this mistake - set [color] effect to 100 was intended to read as "go white" but Scratch's color effect actually rotates hue around the color wheel, so it could never produce white on a colored sprite - confirmed via web search and fixed (switched to a genuine costume swap, matching the white-sprite-swap fix already used in the other seven engines). This is the same mistake Lesson 5's own Sidebar already correctly warns against in the abstract ("Scratch's 'set color effect' ... cyclically shifts hue rather than multiplying against a tint color"), which means the course's own documented understanding wasn't actually cross-checked against every place Scratch code uses that block. Needs a dedicated pass through Lessons 1-12's Scratch content specifically looking for any other "set [color] effect" usage that assumes it behaves like a tint/blend (going to, or away from, a specific target color) rather than a cyclical hue shift. *(found while fixing Lesson 13's Scratch flash mechanism; not yet audited elsewhere)*
- ☐ Book-version fix candidate: Scratch's Lesson 13 double-kill race condition. Robert reports occasionally destroying two enemy clones with a single bullet (rare, but reproducible), and when it happens, score/enemies_remaining only register ONE hit - the exact symptom of the Stalemate bug returning, caused by the Lesson 3 decision to let the bullet keep traveling up-screen rather than destroying it on its first hit (so one bullet can still be "touching" a second overlapping enemy clone on a later frame/tick before either clone has deleted itself). Robert does not want this fixed in the live course now - noted for the book version only. Likely easy fixes worth evaluating when the book retrofit reaches Lesson 13: disable the bullet's own hit-detection after its first hit (a per-bullet "already_hit" flag, or delete/hide the bullet on its first touch like most of the other 7 engines already do) so it physically cannot register a second kill; or, if keeping the travel-through behavior is load-bearing for some other lesson's design, have each Enemy clone disable its own hitbox/touching-check the instant it's hit (consistent with Unity's Lesson 13 fix for the analogous problem - other.enabled = false on the Collider2D - and Godot's Hitbox.set_deferred("monitoring", false)) rather than changing the bullet. Worth cross-checking whether any of the other 7 engines share this same exposure once the bullet is allowed to keep moving past a hit, or whether Scratch is uniquely vulnerable because of its clone-based, no-unique-ID collision model (already documented elsewhere in this file as the reason Scratch's hit-detection ownership differs from the

other engines). *(reported by Robert during Lesson 13 Scratch wrap-up; explicitly deferred to the book version, not fixed now)*

Notes & Commentary

Freeform observations worth remembering for when the book itself gets written - narrative angles, good anecdotes, structural ideas, and things that only became obvious by actually building the course. Seeded initially by pulling from the project's history up through the Unreal Lesson 2/3/4/13 rebuild and the Cross-Engine Consistency Reference document.

GML Visual: Set Instance Variable can't target a dynamically-created instance: Robert tested Lesson 5's GameMaker Visual color-switch instructions hands-on and found the originally-written steps didn't work: they called for dragging Create Instance's output into a "Set Instance Variable" block's Target field, but that block's targeting dropdown only actually offers Self, Other, or a specific object asset (which recolors every existing instance of that object, not just the newly spawned one) - there's no option to target an instance held in a variable at all. The fix he found and confirmed working: give Create Instance its own Target field (a new local variable, e.g. new_enemy), then use the generic "Assign Variable" block with a dot-notation Name field (new_enemy.image_blend) to reach that one specific instance. Corrected in Lesson 5's Handout and Key, and added as a named exception to Standards Rule 4 so a future contributor doesn't reintroduce the same mistake by over-applying "prefer the purpose-built block." Checked Lessons 3 and 13's similar-looking Set Instance Variable uses (targeting Self and Other, respectively, inside a Collision event) and confirmed both are legitimate - Self/Other are real, supported targets on that block; a Create-Instance-captured variable is the specific case it can't reach. No broader retrofit needed beyond Lesson 5 based on this check, but worth re-verifying if a similar spawn-and-tint pattern shows up in a not-yet-audited lesson.

Why 8 engines: The multi-engine structure (one shared weekly lesson, built out independently in Scratch, Godot, Unity, GameMaker, GDevelop, TIC-80, 001 Game Creator, and Unreal) is probably the book's core hook - it's an unusual structure for a game-programming book, most of which pick one engine. Worth an early chapter making the case for why explicitly: it demonstrates that the underlying concepts (loops, events, state, collision) are engine-agnostic, and lets a reader/student pick whichever engine matches how they think (text code, blocks, or pure visual wiring) without missing anything.

The Level 1/2/3 framework as the book's spine: MVP / Complete / Refined is used every single week for a small exercise, AND re-applied at the whole-semester level in Lesson 15 (MVP = Lesson 6, Complete = Lesson 11, Refined = Lesson 15). That fractal quality - the same rubric working at two very different zoom levels - is a strong structural device for the book itself, not just the course.

The Structure goal as a running architecture throughline: Every lesson's Structure goal is doing the same underlying job (separation of concerns) but the book should resist presenting it as one repeated checkbox - it genuinely escalates week to week: co-locate-by-trigger, then object-owns-its-own-behavior, then spawner-vs-behavior, then unrelated-counters-stay-unrelated, then single-reusable-entry-point (DRY), then consolidate-a-system-built-piecemeal. Full breakdown already captured in Cross_Engine_Consistency_Reference.docx - cite it rather than re-deriving this in the manuscript.

Rule 5 origin story is good anecdote material: The Unreal Lesson 2/3/4/13 rebuild this session is a genuinely good illustrative story for a sidebar or pull-quote: Unreal's content had

quietly drifted into flat-color Materials where every other engine used the real supplied sprite, close enough to work but not actually consistent - which is exactly what prompted Robert to articulate the Engine Deviation rule (Standards Rule 5) explicitly. Good real example of "a rule written after the mistake it prevents," which is honest and relatable for readers building their own multi-platform anything.

Engine-specific gotchas as troubleshooting sidebar content: Several hard-won, engine-specific bugs surfaced this project are strong candidates for "common pitfalls" sidebars rather than being filed away as pure production trivia: Unreal's Blank template auto-possessing a flying DefaultPawn and eating movement input; the "multiply a color parameter by white is a no-op" trap that bit Godot, Unity, GameMaker, AND Unreal independently for the Lesson 13 hit-flash; GameMaker/001GC's generic "destroyed" hook firing for both a hit AND an escape in Lesson 7, requiring an actual code restructure. These read well as "here's a mistake you'll probably also make, and why it happens" content.

Production philosophy worth explaining to readers, not just following internally: The Self-Containment rule (no lesson sends a student hunting for an asset mid-class) and the Purpose-Built-Blocks rule (use the engine's specific named block over a generic one, in visual scripting) both read as genuine craft opinions a book reader would want stated outright, not just inferred from the finished lessons. Consider a short "how these lessons were actually built" chapter or appendix drawing on Lesson_Production_Standards.docx.

Source documents - don't duplicate, cite: Lesson_Production_Standards.docx and Cross_Engine_Consistency_Reference.docx (both in Lessons/) are the primary source material for anything about course philosophy, cross-engine rationale, or production conventions. This notes document should stay focused on book-specific decisions, open questions, and raw commentary - not a third copy of the same rules.

Rule 6 retrofit scope: forward-only vs. retrofit: Standards Rule 6 (SuperStretch tie-in, a Refinement and Style pass, and visual-scripting node-reuse guidance) is a standing requirement now, but it wasn't in force when Lessons 1-15 were built, so none of the 15 completed Handouts/Keys currently carry it. Two options: (1) retrofit - go back through all 15 lessons x up to 8 engines and add Rule 6 content to the live, already-shipped Handouts/Keys now; (2) forward-only - apply Rule 6 starting with Lesson 16, and write the Rule 6 content for Lessons 1-15 fresh at book-writing time instead, since the book chapter is a separate deliverable from the live-class Handout and can carry more reflective, after-the-fact content anyway. Leaning recommendation: forward-only. The SuperStretch tie-in is cheap to backfill later - most of that content already exists in the standalone SuperStretch docs and just needs pulling into context. The Refinement-and-Style and node-reuse content is genuinely new writing, at a scale (15 lessons x up to 8 engines) that would be a large side project for a live class with no real classroom benefit, since the students who used Lessons 1-15 have already moved past them. Better to write that material once, well, when the manuscript actually reaches those chapters, than twice - once retrofitted into a Handout already taught from, and again for the book. This is a recommendation, not a decision; Robert asked to scope this only and explicitly did not authorize starting the retrofit.

"Who's in charge" - a real Rule 6 case study, deliberately not retrofitted: Lesson 4's shipped Handout/Key (every engine) has one side of a collision - the Bullet in Unity/GameMaker/GDevelop/TIC-80, the Enemy in Godot - detect the hit and destroy BOTH objects. The cleaner alternative: Bullet and Enemy each detect their own hit independently and destroy only themselves, so neither object needs to reach into and manipulate the other. Robert built his own project this way and confirmed the decision explicitly: keep the simpler, common

beginner pattern in the live Lesson 4 content on purpose (functional progress first, for a student still getting collision detection working for the first time), and save the "who's in charge" framing - already introduced verbally in class, not yet written down anywhere - for a Rule 6 Refinement and Style section or a dedicated Sidebar later. Good, concrete, real-classroom example of Rule 6 in action: recognizing a legitimate improvement mid-lesson and consciously choosing NOT to interrupt functional progress with it. A companion example from the same conversation: Unity's Level 3 Sound idea already avoids a sound getting cut off by Destroy() via AudioSource.PlayClipAtPoint() (a detached, self-cleaning audio source) rather than Godot's hide+await+queue_free() dance - Robert had used a plain AudioSource as a child of the object instead, which gets torn down along with its parent even though Destroy()/queue_free() are deferred to end-of-frame, not instant. Same root cause as the collision-detection case (an object's behavior depending on something outside itself), same fix (self-contained ownership) - a nice paired example for the same Sidebar or Refinement section.

Diagram pilot: Lesson 5 flowchart + GameMaker node wireframe: First use of hand-drawn branded diagrams outside Lesson 2's original set, prompted by Robert noting he keeps re-explaining the same logic verbally in class. Two new diagrams, built as branded SVG (matching the existing Brand_Assets/Diagrams navy/gold UML style) and rendered to PNG via cairosvg: diagram_spawn_loop.png, an engine-agnostic activity diagram of the counter/repeat-check/create/increment shape behind every engine's Level 1 solution this lesson, embedded into all 8 engines' Handout AND Teacher's Key (right before the Level 1 steps/code); and diagram_gml_visual_wireframe.png, a GameMaker-only mock-up of the actual GML Visual block layout (Instance - Create Instance's Target field feeding Variable - Assign Variable's dot-notation Name field), embedded into GameMaker's Handout Level 3 Visual-Track section and Key altNotes section only. Scoped this way per Robert's explicit answers: diagrams live in both the Handout and the Key (not just one); this is a pilot on Lesson 5 only, not yet a retrofit of Lessons 1-4/6-15 and not yet a standing Standards Rule; and node-wireframe diagrams are GameMaker-only for now, since it's the only engine with students actively switching between Script and Visual tracks - Unreal was deliberately excluded from wireframe scope since Robert's two Unreal students recently moved to lighter-weight engines. See the new ToDo item for the retrofit/standing-rule decision, which is intentionally not made yet.

Diagram pilot promoted to Standards Rule 9 - full Lesson 5 extension: Robert decided the pilot's outcome directly: diagrams become a standing rule (Standards Rule 9), not a Lesson 5-only experiment, and he expanded the scope in the same request rather than narrowing it. The Activity Diagram is now standard in every lesson's Handout, Key, AND Slide Deck (previously Handout+Key only). Engine node diagrams now cover all three dual-track engines - GameMaker, Unity, and Unreal - reversing the earlier GameMaker-only wireframe scoping once Unreal's own student-enrollment status turned out not to matter for whether the diagram gets built. The Key always gets a node diagram (Level 1, plus a full combined-sequence diagram for Level 3 when the visual path is complex enough); the Handout always gets one for Level 1 specifically (added as an explicit follow-up note - previously Handout diagrams were novelty-gated only), plus an additional one when something genuinely unusual is introduced beyond Level 1 (the GameMaker back-door wireframe remains the model case for that half of the rule). Sidebars and Slides get diagrams opportunistically, when they'd clarify the text, not as a fixed requirement. Robert also flagged that a Slide Deck's diagram of a concept may deliberately not match an engine-specific document's diagram of the same idea, since engine-specific diagrams must authentically reflect that engine's actual visual paradigm (GameMaker's solid block-stack vs. Unity/Unreal's node-graph-with-pins) while Slides can simplify or generalize. Four new diagrams were built to extend Lesson 5 under the new rule: diagram_gml_visual_level1.png and diagram_unity_visual_scripting_level1.png (Level 1-only node diagrams for the Handouts), and

diagram_unreal_blueprint_level1.png plus diagram_unreal_blueprint_full_sequence.png (Unreal's first node diagrams in this course, for both Handout and Key). All four wired into build_lesson05_handouts_keys_checklist.js's data-driven diagram hooks; the generic diagram_spawn_loop.png was added to the Slide Deck's Loops section; and diagram_unreal_blueprint_full_sequence.png plus the existing diagram_gml_visual_wireframe.png were added to the Lesson 5 Sidebars' "Script vs. Visual Coding" topic to reinforce its existing Unreal-vs-GameMaker instance-reference discussion. See Standards Rule 9 for the formal rule text and the ToDo items above for the still-open past/future retrofit work.

Correction: Apply To solves the GML Visual targeting problem after all: Robert found, through his own hands-on testing in GameMaker, a cleaner solution to the exact problem the earlier "GML Visual: Set Instance Variable can't target a dynamically-created instance" note (above) documented - and it changes that note's conclusion. The earlier note's finding that Set Instance Variable's Action Scope dropdown can't reach a variable-held instance reference was correct as far as it went, but its fix (a generic Assign Variable block with a typed dot-notation Name field, new_enemy.image_blend) turns out not to be necessary. The actual solution: drag an "Instance - Apply To" block right after Create Instance, and attach whatever needs the scoped target - an If/Else block, in this lesson's case - to the RIGHT of Apply To, not below it, which is what nests it inside Apply To's scope rather than just sequencing after it. Apply To's own Action Scope pop-up has an Expression field at the top, separate from its Self/Other/object list, and typing the same variable Create Instance's Target field captured (new_enemy) into that field re-points what "Self" means for every block nested inside. That means the actual purpose-built block - Set Instance Colour, not the more generic Set Instance Variable - works perfectly once nested inside Apply To's scope, with no generic block or typed workaround needed at all. Corrected in Lesson 5's Handout and Key, in Standards Rule 4 (the "confirmed exception" is now a resolved case), and in the Sidebars' Script vs. Visual Coding topic, which had built an entire GameMaker-vs-Unreal comparison around GML Visual allegedly having "no socket" for an instance reference - Apply To's Expression field is exactly that socket, just reached through a scope-setting block and a pop-up field rather than a draggable wire on the canvas. Both diagrams (wireframe and full sequence) were redrawn to match. Worth a general lesson for future audits: a purpose-built block's own targeting dropdown looking limited doesn't necessarily mean the block can't be used - check whether a scope-setting block like Apply To (or an engine's equivalent) changes what's available before concluding a generic/typed fallback is the only option.

Ternary operator pulled from Lesson 5's GameMaker code; save it for Semester 2: Lesson 5's GameMaker Code path originally set image_blend using a one-line ternary, (i % 2 == 0) ? make_color_rgb(200, 25, 55) : make_color_rgb(230, 100, 130) - a pattern Robert genuinely likes and uses himself all the time, but judged a bit much to introduce to beginning students this early in the course. Rewritten as a plain if/else block instead, which is longer but reads unambiguously for a student who hasn't met conditional expressions as values yet. Robert's own suggestion: the Semester 2 capstone project - once students are comfortable with the fundamentals and are extending a more mature codebase - is a good, deliberate place to introduce the ternary operator formally, rather than leaving it to be picked up incidentally or never. Not yet scheduled to a specific Semester 2 lesson; see the paired ToDo item.

Demo-engine rotation drifted, then got re-planned as Standards Rule 10: Robert noticed Godot had been the live lecture-demo engine every single lesson from 3 through 14 (not just a few weeks running, as he'd first suspected) and asked for a full per-lesson list, which surfaced the drift. A scan back through the project turned up real evidence that a rotation had genuinely

been pre-planned, not just remembered inaccurately: the Syllabus, Engine & Platform Guide, and Engine Showcase docs all promise students an even rotation across engines/styles, and the 30-Lesson Master Schedule spreadsheet has an actual Featured Engine + Coding Style column for all 30 lessons modeling that balance. The promise was simply never enforced as a Standards Rule, so nothing stopped a lesson's "continue last week's demo project" framing from perpetuating whatever engine got picked once. Since Lessons 3-15's actual topics have diverged from the master schedule's original per-week order, the old schedule couldn't just be restored - a new rotation had to be designed against the lessons as actually built. Robert then narrowed the pool to only the engines/styles with active students (Scratch, GameMaker script, GameMaker visual, Unity C#, Godot), deferring full 8-engine/3-style balance to a ToDo item (above) rather than designing for engines nobody's using yet. Finalized rotation for Lessons 5-14, each choice appearing exactly twice, no back-to-back repeats, preserving a "same engine, two modes" GameMaker callback pairing (mirroring the original schedule's design intent): L5 GameMaker Visual (+ brief script contrast, ties to the Script-vs-Visual Sidebar), L6 Scratch, L7 Godot (unchanged), L8 Unity C#, L9 GameMaker script (callback to L5), L10 Godot (unchanged), L11 Unity C#, L12 GameMaker Visual (callback to L5), L13 Scratch, L14 GameMaker script (callback to L9). L15 stays demo-free (workshop/showcase day) per the original design. Every affected lesson's Plan (Featured Engine field), Teacher's Script (demo cues and talking points), and, for Lesson 5 only, Slide Deck speaker notes were rewritten to be technically accurate to their newly assigned engine - not just relabeled.

Enemy-vs-enemy self-collision: a real bug, found engine by engine, fixed quietly rather than taught yet: Robert hit a genuine runtime bug testing Lesson 5 Godot himself: no enemies appeared on screen or in the Remote Scene tree during play. Diagnosis (his own testing plus a review of the shared Hitbox/area_entered collision architecture from Lessons 3-4) found the real cause: Godot's Hitbox area_entered handler has no type-check on what triggered it, and at the original 40px-apart / 60px-spacing layout with 128x128px sprites, adjacent Enemies' hitboxes overlapped at spawn and mutually destroyed each other before the first frame was ever seen - confirmed directly by Robert reducing the loop count and watching enemies survive, then hearing the destroy sound fire once the count went back up. His chosen fix, deliberately the narrow one: widen spacing only (Vector2(150 + i * 200, 50)), explicitly declining the deeper architectural fix (a real Collision Layer/Mask setup, or a type-check in the handler) for now. Robert then reported Scratch has the identical problem - overlapping clones causing double-hits rather than full destruction - which he'd already independently fixed himself (enemy_i renamed to enemy_number, spacing formula changed to enemy_number * 100 - 300) before this was ever raised as a course-wide question. That prompted a full audit of the remaining six engines for the same generic-trigger-fires-on-any-overlap pattern: GameMaker is architecturally immune (its Collision Event is object-pair-specific, not generic); Unity (OnTriggerEnter2D + CompareTag), Unreal (Cast To BP_Bullet), GDevelop (named-pair collision condition), and TIC-80 (hand-coded rectangle-overlap math) are all safe by design for the same reason. Only 001 Game Creator was found vulnerable (its "Collided with actor" trigger is generic, with no actor-type check) - fixed the same way as Godot and Scratch, by strengthening the placement instructions in Lessons 5 and 14 ("at least a full enemy-width of empty gap") rather than restructuring the trigger logic. Robert was explicit about scope throughout: fix the practical problem silently, everywhere it exists, right now - but do not yet teach collision-type filtering (Bullet vs. Player vs. Enemy vs. Enemy) as a formal topic. A review of Lessons 6-15's actual content confirmed there was no orphaned teaching moment being skipped by this choice: true Player-vs-Enemy touch collision never appears anywhere in Semester 1 (only Bullet-vs-Enemy from Lesson 4, and an escape-based life-loss mechanic in Lesson 8), and Lesson 15's own teacher script already frames collision-type filtering as Semester 2 territory. Good candidate for a Semester 2 sidebar or lesson-opener later: Robert's own framing for it, already used verbally

in class per his message, was "Now our enemies are moving around, they can collide with each other AND the player. How do we fix this? What even is the right action we want for each type of collision?"

Cross-engine consistency promoted from habit to Standards Rule 12: While retrofitting Unity's Lesson 5 code to Rule 11 (meaningful variable names), a real naming conflict surfaced: Unity's EnemySpawner already has a public enemyCount field (the total number of enemies to spawn), so reusing that same name for the loop counter - the literal move Rule 11 alone would suggest, matching GameMaker and Godot's enemy_count - would shadow it, technically legal in C# but genuinely confusing. The loop counter was named enemyIndex instead, a distinct and still meaningful name, and the mismatch was flagged to Robert rather than resolved unilaterally. Robert's response went beyond just answering that one question: leave the enemyCount/enemyIndex mismatch alone for now, but treat the underlying question - keeping variable names AND code structure consistent across all 8 engines - as a fundamental precept of the course, not a nice-to-have. His reasoning, stated directly: he has personally observed students helping each other across engine and coding-style lines - a student on one engine comparing notes with a student on another, working through the same week's exercise - and the more common ground the actual names and structure share, the easier that peer help is. A student has to mentally translate "my enemy_count" to "their enemy_number" before they can even start comparing approaches; consistent naming and structure removes that translation tax entirely. This was true from the start of the multi-engine design but wasn't enforced strictly early on (Lessons 1-4 predate any naming rule at all, and Lesson 5's own Rule 11 fixes were applied engine-by-engine as each was caught rather than as one coordinated pass) - Robert is now retroactively naming it as something the course has valued implicitly all along, now that enough real classroom evidence exists to state it as a rule outright. Formalized as Standards Rule 12; a dedicated future cross-engine reconciliation pass (covering both naming and structure, across all lessons and engines) is tracked as a ToDo item above, deliberately scheduled later rather than forcing an immediate resolution of the Unity case that surfaced it.

Data Dictionary project launched to drive the Rule 12 consistency goal: Immediately after Rule 12 was formalized, Robert asked for a central, formal Data Dictionary - not just a ToDo describing the consistency goal, but an actual working reference naming every variable, function, and file the course creates: one common cross-engine concept name (e.g. "per-frame update"), paired with each engine's own actual name for that same concept where engines can't literally share one identifier. Undecided yet whether this will ever be shown to students, or stays a production-only tool - built either way, since it's useful internally regardless. Robert's first-choice format was a genuine Microsoft Access database with a data-entry Form and a Report, explicitly because he dislikes working in spreadsheets. Checked before building anything: Access's .accdb format (2007+) has no Linux-native authoring path at all - mdbtools, the standard Linux toolkit for Access files, doesn't even read/write the modern .accdb format (it's limited to the legacy Jet 3/4 .mdb format from Access 97/2000/2002), and more fundamentally, Access Forms and Reports are proprietary, VBA-hosted application objects with no file-level equivalent outside Access itself - no export format, no library, nothing short of the real Access application (Windows-only, not available in this sandboxed environment) can author a working Form or Report object, regardless of which .mdb/.accdb table format is targeted. Confirmed via both a direct tool check and a web search before ruling it out, rather than assuming. Robert had already pre-approved the fallback ("I'll live with an Excel spreadsheet but I'll pout about it"), so the dictionary is being built in Excel instead, with an explicit effort to make it feel less like a bare spreadsheet than the default - data-validation dropdowns, a proper per-engine reference layout, and the closest practical approximation of the Form/Report feel Excel supports. Same conversation also raised the open case/style question (PascalCase vs. camelCase vs.

snake_case) as a Rule 12 sub-decision, with Robert's stated leaning toward prioritizing cross-engine consistency in the chosen case style over each engine's own local idiom, explained via Sidebar wherever the two disagree - see the paired ToDo entry, not yet a final decision.

GameManager: a dedicated shared-state object, promoted from a Unity-only accident to Standards Rule 13: Reviewing Lesson 7 surfaced an asymmetry that had been sitting unnoticed since Lesson 6: Unity's Handout/Key already routed the win-counter (enemies_remaining) and win-flag (has_won) through a small dedicated GameManager object, while every other engine had that same shared state living wherever it happened to land - the spawner in some engines, a collision branch in others. This wasn't a deliberate Unity design choice being withheld from the rest of the course; it traced back to how Unity's Lesson 6 content happened to get written, and nobody had asked "why does only one engine do this" until Robert did. Once asked, the answer was obvious: a dedicated object whose only job is holding game-wide information is the better habit, full stop - Unity had simply stumbled into the right pattern first. Formalized as Standards Rule 13 (a dedicated manager object for any state more than one other object needs to read or change), and retrofitted into Lesson 6 across Scratch (a hidden GameManager sprite driven by broadcast/receive), Godot (a child GameManager Node reached via signal connection), GameMaker (obj_gamemanager, a placed instance), and Unreal (BP_GameManager, a placed Actor with a dispatcher binding) - Unity needed no change, having already modeled the target pattern. Good real-course illustration for the book of a rule discovered by noticing an inconsistency rather than designed up front - the opposite direction from Rule 5's origin story (a rule written after catching a mistake), this one was a rule written after catching an accidental correct answer nobody had generalized yet.

Lesson 7 compliance pass: GameManager routing, naming drift, and the full diagram/prose standard: With Rule 13 in place, Lesson 7 itself needed the same GameManager routing applied to its own win-tracking logic (all engines), plus a Teacher's Script callout telling students to go retrofit their own Lesson 6 project before starting Lesson 7's lab work, so the fix lands in every student's actual codebase rather than staying a documentation-only change. Auditing Lesson 7 against the full current Standards Rule set while making that change turned up two smaller drifts worth recording: Rule 11 (meaningful names) had regressed in a couple of spots - Scratch's enemies_left/enemy_i and Godot/GameMaker/Unity's bare speed field all got renamed to match the enemies_remaining/enemy_number/enemy_speed conventions established elsewhere - and Rule 9 (diagrams) hadn't been applied to Lesson 7 at all yet. Five new diagrams were built and wired into every Handout, Key, and the Slide Deck: a generic Activity Diagram contrasting the two ways an enemy can disappear (escape vs. hit, only one of which should decrement the counter - the exact bug this lesson's exercise centers on), a State Diagram for the enemy's two possible endings, and Level-1 node diagrams for the three dual-track engines (GameMaker GML Visual, Unity Visual Scripting, Unreal Blueprint). Unity's Visual Scripting altPath prose was also tightened afterward to name each node exactly as the diagram draws it (Update -> Translate -> Branch -> Destroy) rather than compressing the graph into a single dense sentence, matching the per-node granularity Lesson 5 set as the specificity bar for alt-path writing. Unreal's Blueprint path needed no equivalent prose tightening - it was already written at full specificity as the lesson's PRIMARY path (not an altPath), and the audit's original complaint about it (no diagram backing the prose) was resolved by the new diagram set.

Lesson 8 compliance pass: GameManager retrofit, naming fixes, and the full diagram/notes standard: Lesson 8 (the lives/lose-condition lesson - the mirror image of Lesson 6's win condition) got the same treatment as Lesson 7, this time against a slightly different set of gaps. Rule 13: Lesson 8's lives counter and lose-flag (has_lost) were retrofitted onto each engine's GameManager object (mirroring exactly how enemies_remaining/has_won

already route through it since the Lesson 6/7 retrofit), rather than living on the player or a collision handler directly - the same routing pattern, applied to the opposite condition. Rule 11/12: a genuine TIC-80 naming regression was caught and fixed (the bare j/e loop variables flagged as a known gap in the standing ToDo list finally got their Lesson 8 instance renamed to enemy_index/enemy, matching the Lesson 7 precedent), alongside a small Scratch naming drift back toward an earlier inconsistent variable name. Rule 7: GameMaker's Draw GUI altPath and Unity's Visual Scripting altPath both had field-value and node-name gaps tightened to full explicitness (exact Draw Text field values spelled out; Unity node names matched one-for-one to what the new diagram draws), closing the same category of gap Lesson 7's Rule 7 audit had already fixed elsewhere. Rule 9: a full diagram set was built and wired in - one generic Activity Diagram (the lose-condition logic, shared across every engine's Handout/Key/Slide Deck) plus three engine-specific Level 1 node diagrams (GameMaker GML Visual, Unity Visual Scripting, Unreal Blueprint), wired into every Handout and Key. This lesson surfaced a placement nuance not previously written down: in the Handout, the node diagram nests under the altPath for engines whose primary documented path is code (Unity, GameMaker), but sits as a top-level diagram for Unreal, whose primary path is Blueprint itself, not an alt; in the Key, the node diagram is always top-level for all three dual-track engines, since the Key presents the full combined solution rather than a split main/alt track. Rule 10/8: the Slide Deck had drifted to reference Godot in its live-demo materials list and Slide 9's content box, left over from before the Lesson 5-14 rotation was finalized (Lesson 8's assigned engine is Unity C#) - corrected to Unity/GameManager.RegisterEscape() throughout - and had zero addNotes() calls despite Robert's classroom now running Presenter View, so six speaker-notes entries were added (slides 5, 7, 9, 11, 13, 14 - matching the Lesson 6 reference pattern of skipping title/overview/divider slides), each carrying that slide's matching Teacher's Script section verbatim, including bracketed stage directions and timing checkpoints.

Lesson 9 compliance pass: score joins the GameManager, a Unity Rule 7 gap, and the full Rule 9 diagram/Rule 8 notes treatment: Lesson 9 ("On the Scoreboard" - score tracking plus a persistent on-screen HUD, introducing the Push vs. Poll display-update concept) got the same full audit-then-fix treatment as Lessons 7 and 8, requested proactively by Robert while he personally reviewed Lesson 8's materials rather than in response to a specific bug report. Rule 13 was the single biggest finding: Lesson 9 had fully regressed on the GameManager pattern across four engines, each in a different way - GameMaker was routing score through obj_spawner instead of obj_gamemanager; Scratch had the Enemy sprite's own script changing score directly instead of staying confined to GameManager's existing "when I receive [enemy hit]" broadcast handler; Godot had score and a new update_hud() function living on "the main scene's script" instead of GameManager's script, breaking the established get_node("../ScoreText")-from-GameManager pattern; and Unreal had Score and the HUD widget setup living on the Level Blueprint instead of BP_GameManager, the same Actor that already owns EnemiesRemaining and Lives. All four were corrected to route score (and, for Godot/Unreal, the new push-based update function) through the established manager object, cross-checked line-by-line against Lesson 8's already-compliant content as the reference baseline for each engine's correct object/function names rather than improvising a new pattern. Two smaller drifts were caught and fixed opportunistically along the way: a Rule 11/12 Scratch naming regression (enemies_left had crept back in, corrected to enemies_remaining), and a Rule 7 gap in Unity's Visual Scripting altPath (vague "a node" references tightened to exact node names and categories - Format String/String category, Add/Math-Scalar category - matching the specificity bar Lessons 5 and 8 established). Rule 9: a full diagram set was built - one generic Activity Diagram ("Score Joins the Hit Path - Then Push or Poll Takes Over," covering both the GameManager routing and the Push-vs-Poll architectural choice in one picture) plus three engine-specific Level 1 node diagrams (GameMaker GML Visual, Unity

Visual Scripting, Unreal Blueprint) - and wired into every Handout and Key following the Lesson 8 precedent exactly: nested under the altPath in the Handout for GameMaker and Unity (whose primary path is code), top-level in the Handout for Unreal (whose primary path is Blueprint), and always top-level in the Key regardless of Handout placement, since the Key shows the combined solution. Rule 10/8: the Slide Deck's materials list and Slide 9 content box had drifted to reference Godot, left over from before the Lesson 5-14 rotation was finalized (Lesson 9's assigned engine is GameMaker script, a deliberate callback to Lesson 5's GameMaker Visual demo) - corrected to GameMaker/obj_gamemanager.score throughout - and had zero addNotes() calls, so six speaker-notes entries were added (slides 5, 7, 9, 11, 13, 14, the same pattern Lessons 6 and 8 established), each carrying that slide's matching Teacher's Script section verbatim.

GameMaker drag-and-drop uses instance variables (__dnd_lives, __dnd_score) - a correction found by testing: While reviewing Lesson 8 for GameMaker, Robert noticed that the Visual Scripting content relied on deprecated built-ins: the old global lives/score/health have been replaced by drag-and-drop's own instance-level variables, __dnd_lives and __dnd_score (dnd = drag and drop), and purpose-built blocks exist for them (Set Lives, If Lives, Draw Instance Lives, Set Score, Draw Instance Score), each with a Target drop-down that must point at the GameManager object. The earlier content had wrongly used generic Set Instance Variable blocks with Lives/Score as a supposed recognized built-in. The first proposed fix (keep a custom variable in code and leave the code and block paths separate) was rejected in favour of making the code path use the very same __dnd_ variables, so a student can mix typed code and blocks in one project. Lessons 8 and 9 were corrected on that basis; Lesson 8 also gained the missing sound assets (sfx_lifelost, sfx_gameover, sfx_start) so students never have to find their own. Good book anecdote: the author's own hands-on testing found what documentation-driven drafting missed, and the same session surfaced follow-up questions (dotted obj.variable vs. Target drop-down, Prefabs, asset-name prefixes) now tracked in the To-Do list.

GML Visual has no Else If block - a genuine engine-specific gotcha, not yet retrofitted: Robert's observation: GameMaker's GML Visual (drag-and-drop) toolbox appears to have no "Else If" node/block at all, unlike the code path's natural else if chain. The only way to express a third (or later) branch of a multi-way decision in Visual is to nest another If-style block one level deeper, inside the previous block's Else slot - so a three-way decision isn't three same-level blocks in a row, it's an If, whose Else contains another If, whose Else contains the final branch, and so on for each additional case. This is exactly the kind of engine-specific gotcha the project has already flagged as good troubleshooting-sidebar material elsewhere (see the earlier note on engine-specific gotchas as a candidate for a "common pitfalls" chapter): a student who's only ever written else if in text code will very reasonably go looking for an Else If block in GameMaker Visual, not find one, and assume the tool can't do what they want rather than realizing the shape is just different. Not yet checked against the shipped course content - see the matching To-Do item for which lessons (Lessons 8, 9, and 11 are the likely candidates, anywhere a lesson's logic needs to tell apart three or more outcomes - lives hitting zero vs. enemies hitting zero vs. neither, or a wave clearing vs. the whole round being won) actually have a three-or-more-way branch in their GML Visual content, and whether that content already nests correctly or was written as if a flat Else If existed.

Debugging 101 / Testing 101 / Testing-Debugging 201 - a companion course family, spitballed from this project's own bug history: Robert's idea, raised as a sideways thought rather than a scoped proposal: a new book/class, "Debugging 101," built around a code repository pre-loaded with a variety of deliberately planted bugs for students to find and fix. The named bug categories he gave map with unusual precision onto bugs this very project actually

produced and recorded: flat-out broken code that doesn't compile or run; a failed edge case (the Stalemate is exactly this - kills and escapes are each handled correctly in isolation, but the combination where some of each happen in one round was never considered); a new feature breaking an existing one (the dropped Level 3 sounds - a Rule 14 "complete method" regeneration for a later lesson silently deleted working Level 3 code from an earlier one); a mismatch between a design doc and the actual code (the WaveAnnouncementText/WaveText naming mismatch between a Handout's prose and its Teacher's Key); a broken refactoring (the GameMaker Visual targeting saga - a generic block was swapped for another generic block, then finally a purpose-built one, with two of the three attempts subtly wrong in a way only hands-on testing surfaced); and a misinterpreted feature (built to a literal reading of a spec, but not to what was actually intended - the enemy-vs-enemy self-collision bug is a plausible fit here too, where a generic "anything touched this" trigger was technically working as coded but not as meant). The parallel idea, "Testing 101": the inverse situation - an existing codebase plus a design doc, but no test suite at all, where the student's job is to write the tests. A possible synthesis, "Testing/Debugging 201": combine both so that writing a test case IS the mechanism that proves a fix actually worked, not just a separate skill taught alongside debugging - red test before the fix, green test after. None of this is scoped: no decision yet on audience (same Middle/High School homeschool audience as Video Game Creation, or a different one entirely), language/engine (the bug-category list reads as language-agnostic, but the actual repository would need a concrete stack), length (a standalone book, a short course, a unit folded into an existing one), or relationship to the main course (standalone companion vs. a Semester 2/Year 2 extension of Video Game Creation itself, given how directly its source material comes from this project). Recorded here purely so the idea isn't lost before it's had time to develop; revisit once Semester 1 of Video Game Creation is further along or complete.

Lesson 12 compliance pass: a real cross-engine sound-architecture conflict, resolved in favor of Option 1: Auditing Lesson 12's shipped materials (requested by Robert before review) surfaced a genuine design-decision conflict, not just a drift-style bug like Lessons 7-9's compliance passes found. The Handouts/Keys had Godot and Unity building a brand-new shared sound player this lesson - Godot's RoundSound AudioStreamPlayer2D swapping its .stream property between three clips, Unity's roundAudioSrc AudioSource with a brand-new GameManager.instance singleton calling PlayOneShot() - while Scratch and GameMaker simply added more one-shot sound calls to existing branches, no shared player at all. This wasn't a copy-paste typo: it directly contradicted the dedicated-per-event sound pattern Lessons 6 and 8 had already established and shipped in every engine (WinSound/LifeLostSound/GameOverSound as separate AudioStreamPlayer2D nodes in Godot; winSound/lifeLostSound/gameOverSound as separate Inspector-assigned AudioClip fields reached through the already-existing GameManager.instance in Unity). The Lesson 12 content also introduced a brand-new sfx_lose.wav asset, when sfx_gameover.wav already existed and served the identical game-over moment since Lesson 8 - an unnecessary and unexplained new asset, not a case where two legitimately different sounds were needed. Rather than resolve this unilaterally, Robert was asked directly: Option 1 was to retire the shared-player design and the new sfx_lose asset, folding Lesson 12's wave-clear sound into the existing per-event pattern (one more dedicated node/field, reusing sfx_gameover.wav for game-over exactly as Lesson 8 built it); Option 2 was to keep the shared-player design and treat it as this lesson's deliberate new teaching point (a legitimate, if different, architecture worth knowing), fixing only the naming/wrong-asset issues. Robert chose Option 1 for the live course - see the separate note below for Option 2's preserved reasoning for the book version. All four active engines' (Scratch, Godot, Unity, GameMaker) and all four paused engines' (GDevelop, TIC-80, 001 Game Creator, Unreal) Handouts and Keys were rewritten under Option 1: one new dedicated sound per engine for the wave-clear event (Godot's WaveSound node, Unity's waveClip field,

GameMaker/Scratch/the rest's one new sfx_wave call), and sfx_gameover (and, where applicable, sfx_lifelost) made mandatory rather than optional, reusing the Lesson 8 assets rather than sfx_lose. The Prep Checklist, GDD sound-asset description, Plan, and Teacher's Script were all corrected to match (the Teacher's Script's featured-engine demo narrative, built entirely around the old shared-player idea, needed a full rewrite of its 0:15-0:35 segment). The Slide Deck's zero existing speaker notes (a separate Rule 8 gap, unrelated to the sound-architecture question) were filled in across all eight content slides while the deck was already being rebuilt for the wording fixes. The old Assets/sfx_lose.wav file in the Lesson 12 folder is now unused under Option 1 but was left in place rather than deleted unilaterally - flagged to Robert as an open cleanup decision.

Sound should have been its own dedicated lesson all along - a retrofit decision, and a sharper Level 1/Level 2 framing it surfaced: While getting ready to start on the just-fixed Lesson 12 materials, Robert had a realization he wished he'd had much earlier: Lesson 12 - "Hearing It Happen" - is fundamentally a dedicated sound lesson, and in hindsight the course would have been better served treating sound as a genuine, deliberately-scoped topic from the start, rather than scattering it as a series of small optional Level 3 'add a sound' stretch ideas across Lessons 2, 3, 4, 6, and 8 the way it actually happened. His stated regret, in his own words: introducing sound piecemeal in the earlier lessons 'was a mistake.' His proposed fix for the book version: strip the sound-related optional Level 3 content out of those earlier lessons entirely, and let Lesson 12 be where sound is taught properly, as its own concept with real attention - not as a backlog of previously-half-built features finally getting formalized, which is closer to what the live course's current Lesson 12 does. He left the door open to keeping some sound ideas as earlier-lesson stretch goals if that still makes sense on reflection, but the explicit design principle either way is to make Lesson 12 a genuine 'pull it all together' moment rather than a cleanup lesson. Importantly, this is scoped as book-only: the live course's Lessons 2-11 keep their existing optional-sound Level 3 content as shipped, and this session's Lesson 12 fix (reusing Lesson 6/8's established sounds and adding one new wave-clear cue) remains exactly right for the live course regardless of what the book eventually does differently in the earlier lessons.

A "Sound Manager" as the book-version Lesson 12 refactor target - resolving the scatter-vs-strip question: Robert, reviewing this session's Lesson 13-15 compliance work, raised a resolution to the open "strip sound out of early lessons" question above: "Sound Manager" (sometimes "Audio Manager") is a real, standard game-design pattern - a single dedicated object owning all audio playback (one or more reusable players, volume/mute controls, avoiding the same sound double-firing, sometimes music crossfading), the audio-specific instance of the general Service Locator pattern described in Robert Nystrom's Game Programming Patterns, and one of the most commonly taught "first singleton" concepts in Unity/Godot tutorials. His proposal: rather than stripping the scattered optional Level 3 sound stretch goals out of Lessons 2-8 for the book version (the earlier-recorded leaning), KEEP them - motivated students can experiment with sound early, nothing is lost - and instead make book-Lesson 12 the payoff lesson where whatever a student has scattered gets explicitly refactored into one formal SoundManager object/concept, rather than just adding more sounds to existing branches. Robert's stated reasoning: this gives Lesson 12 more pedagogical significance, turning it into a genuine 'extract into one reusable, centrally-owned thing' lesson - the same recurring teaching motif the course already uses elsewhere (GameManager under Rule 13; spawn_enemies() extracted into its own reusable function in Lessons 10-11; wave_cleared() consolidating two branches in the Lesson 10-11 Stalemate fix) - applied to audio instead of game state or spawning. This also directly revives and sharpens the already-preserved Option 2 reasoning (the shared sound-player design retired from the live course in favor of Option 1, see the

dedicated preservation note above): a formal SoundManager IS essentially Option 2's shared-player idea, now framed explicitly as a deliberate mid-course refactoring lesson (built on top of Lessons 2-8's scattered attempts) rather than a same-lesson architecture choice with no before-state to refactor from. Not decided or scoped - Robert said he'll review this discussion later - but recorded here since it meaningfully resolves the tension between the two previously-recorded, not-yet-reconciled notes on this topic (strip-sound-out vs. preserve-Option-2), and because of its direct kinship with the Rule 13 GameManager pattern and the course's other extract-into-a-manager lessons, it is likely the strongest candidate for how the book ultimately handles Lesson 12.

A sharper Level 1 vs. Level 2 framing, surfaced by the same conversation: Thinking through why scattering sound across early lessons felt wrong in hindsight led Robert to a broader, genuinely useful clarification of what Level 1 and Level 2 are actually FOR across the whole course - distinct from the existing 'Level 1/2/3 framework as the book's spine' note above, which covers the fractal MVP/Complete/Refined structure itself but doesn't address why a student would prefer one level over another. Robert's framing: Level 1 (Minimal/MVP) is specifically a classroom-pacing safety valve - its job is to give a student falling behind a fast, reachable floor so they don't fall so far behind the live class's cadence that they can no longer reasonably absorb the NEXT lesson, not to represent the course's actual intended destination for that week. Level 2 (Complete) is the real preferred target, especially for a self-paced or individually-studying reader (exactly the audience the book serves) - it's a more realistic, complete feature set, and it deliberately blends 'spoon-fed' instruction with 'figure it out yourself' challenge in a way Level 1 alone doesn't attempt to. This directly shapes the sound-consolidation decision above: Robert is fairly sure Level 2 goals should NOT include sound in the earlier lessons once sound is pulled out and consolidated into Lesson 12, since Level 2 is meant to be the realistic, preferred accomplishment for those lessons, and sound doesn't belong in that target picture until its own dedicated lesson actually earns it. Worth writing this Level 1-versus-Level 2 purpose distinction into the book's framework chapter explicitly, rather than leaving a reader to infer it from the numbering alone.

Lesson 15 Complete Codebase deliverable - scoped now, deliberately not built yet: Robert asked for a new Lesson 15 deliverable: a complete, finished codebase per engine (at least the active engines), handed to students at the end of Semester 1 so they have something concrete to check their own work against over the break and come back ready for Semester 2 - distinct from, but a direct realization of, the already-logged "Complete Script Document" Final Deliverable idea. He also raised a second, related complaint while discussing it: GameMaker Visual's node-based logic has been represented as prose description throughout this course so far ("I don't feel like a textual description has been the best approach"), and asked for an actually diagrammatic representation instead. Three scoping questions were asked and answered directly: engines (the active 5: Scratch, Godot, Unity C#, GameMaker Code, GameMaker Visual - the 4 paused engines excluded), depth (Level 3 complete code included wherever it exists, which means backfilling the known Rule 14 Level-3-prose-only gap first rather than settling for the Level 1/2 baseline - chosen specifically because that backfill work also feeds the book), and GML Visual representation (full node-flow diagrams per object, extending the existing Lesson 5/7/8/9/11 wireframe-diagram pilot to cover every object completely, not selective/trivial-skipped). The one firm instruction layered on top of all three answers: don't start building this yet. Robert was explicit that doing it now, before the remaining lessons have been reviewed, would likely be wasted effort - the reviewed lessons could still change in ways that would require redoing the Complete Codebase work. Recorded here as a fully-scoped, ready-to-execute future task, deliberately not started.

"Doing It the Doers of Stuff Way" - why an ongoing Level 4 was abandoned in favor of one final chapter: Robert originally envisioned a fourth tier alongside Level 1/2/3, stretch goals, and Sidebars: something focused on refactoring and on introducing his own particular coding style, applied on an ongoing basis lesson by lesson the way Level 3 and Sidebars already are. Reflecting on this project's own real experience with lesson drift and his own code drift (see the GameMaker Code-vs-Visual parity ToDo, and the general pattern of naming/structure drift this project has repeatedly caught and fixed via Standards Rules 11-13), he concluded an ongoing Level 4 isn't workable in practice, for a specific, practical reason rather than a philosophical one: it forces a choice, every single lesson, between two options he considers both bad. Option A is making the refactor-to-his-style pass a mandatory part of every lesson going forward, which he explicitly does not want - it would slow lesson production and tie every future lesson to keeping a second code path current. Option B is maintaining two genuinely separate, divergent code bases per lesson - the plain Level 3 solution, and a separately-maintained "Refactored Level 3" - which this project has already experienced is a real, recurring maintenance cost (not a hypothetical one), given how much effort has already gone into catching and fixing drift between a single lesson's own Code and Visual paths within ONE engine, let alone an entirely separate refactored version of every lesson across all 8. His resolution: drop the ongoing-Level-4 idea entirely, and instead write a single, dedicated final chapter - "Doing It the Doers of Stuff Way" (a direct reference to his own studio name) - that does the refactoring discussion once, thoroughly, as its own standalone unit, explicitly outside the 30-lesson weekly cadence. Framed as a take-home capstone-adjacent lecture rather than lesson content proper, with an explicit invitation for students to develop their own coding opinions and style rather than simply adopting his - the chapter teaches the ACT and REASONING of refactoring and personal style as a transferable skill, using Robert's own style as the worked example, not as the prescribed answer.

Lesson 13 compliance pass: a severe multi-engine regression, a real Godot/Unity timing bug, and the first Activity Diagram added by student request: Robert asked for a full Standards Rule 1-15 audit and fix pass on Lesson 13, assuming it was "probably on the up and up" after Lesson 12's clean-up - instead, Lesson 13 turned out to carry the worst drift found in any compliance pass so far, likely because it was drafted before several of the GameManager-retrofit and naming conventions established by Lessons 7-12 existed. GameMaker's Level 1 code still called obj_spawner.enemies_remaining/obj_spawner.spawn_wave() throughout, months after Lesson 9's compliance pass had already routed enemies_remaining through obj_gamemanager and Lesson 10's spawn_enemies() rename; TIC-80 had the matching stale spawn_wave() call; GameMaker's altPath still used a generic Set Instance Variable block on sprite_index instead of the purpose-built Set Sprite block Rule 4 requires (the same category of gap the __dnd_lives/__dnd_score correction caught in Lessons 8-9). Scratch's broadcast naming had drifted to Title Case ([Enemy Hit] instead of the shipped convention's lowercase [enemy hit]) during the rewrite and was caught and corrected before shipping. More seriously, both Unity and Godot's Level 1 code had a genuine timing bug baked into the Key itself: GameManager.RegisterKill() (Unity) and destroyed.emit() (Godot) were sequenced AFTER the 0.1-second hit-reaction delay instead of before it, meaning score/wave/win-check would lag visibly behind every hit - directly contradicting this lesson's own central teaching point (the delay should affect ONLY how the enemy's removal looks, never when game state changes). Both were corrected to fire immediately, with only the enemy's actual destruction (Destroy(gameObject) / queue_free()) staying behind the delay. Rule 10: the Slide Deck's live-demo materials list still named Godot instead of Scratch (Lesson 13's actual assigned rotation engine). Rule 8: the deck had zero speaker notes; five were added (slides 5, 7, 9, 11, 13) carrying the Teacher's Script's matching timed sections verbatim. Rule 9: Lesson 13 had no diagrams at all, and Robert specifically asked for an Activity Diagram after students asked about them directly in class - a new generic Activity Diagram ("A Hit Happens - What Runs Now, What

Waits," the first diagram this project has built in direct response to a student question rather than an author-driven audit finding) plus a GameMaker Visual Level 1 node diagram (obj_bullet's revised Collision event plus obj_enemy's new Alarm 0 event, the first diagram to show a GameMaker Visual object gaining a brand-new event rather than just new blocks in an existing one) were built and wired into every engine's Handout and Key, GameMaker's Handout and Key additionally getting the node diagram under its altPath/top-level per the established Lesson 8 placement convention. All affected build scripts (Handouts/Keys/Checklist, Slide Deck) were rebuilt, converted to PDF, and verified via pdftotext that no stale obj_spawner/spawn_wave/Title-Case-broadcast text remained and that both engines' corrected event ordering appears in the shipped text, before shipping to the Lesson 13 folder.

Lesson 14 compliance pass: a stale spawn-routine name drifted into five engines, a genuine Godot/GameMaker ordering bug, and a real Rule 12 discovery (TIC-80 legitimately calls it spawn_wave()): Robert granted standing authorization to audit and fix Lessons 14 and 15 to the Lesson 13 standard while away from the keyboard, to be reviewed together as completed work on his return. Lesson 14's audit found the most widespread single naming bug of any compliance pass: the Handout/Key content called the enemy-spawning routine spawn_wave() in Scratch, Godot, GameMaker, and (correctly) TIC-80 alike - but Lessons 10-13 had already established spawn_enemies() as the real function name in Godot, GameMaker, and Scratch (Unity's SpawnEnemies() and GDevelop/001/Unreal's event-based names were already correct and needed no change). Scratch additionally used enemies_left instead of the shipped enemies_remaining, and - in a near-miss caught only by cross-checking Lesson 11/12's actual canonical broadcast casing - used [spawn wave] instead of the real lowercase [spawn enemies] broadcast name. Investigating TIC-80 specifically to confirm its correct name turned up a genuine, useful discovery: TIC-80 is NOT drifted at all, and actually does call its routine spawn_wave() - a real, pre-existing Rule 12 cross-engine naming inconsistency (TIC-80 says spawn_wave, four other engines say spawn_enemies, Unity says SpawnEnemies) that was sitting undocumented until this audit surfaced it by requiring a definitive per-engine check. Beyond naming, two real structural bugs were caught and fixed: Godot's start_game() resets enemies_remaining BEFORE wave = 1 runs (the opposite order from GameMaker's start_game(), which resets wave before enemies_remaining) - Lesson 14's new wave-dependent formula would have silently used the previous game's leftover wave number unless the Handout/Key explicitly flagged the ordering dependency, which they now do; and GameMaker's Level 1 content had briefly folded the enemies_remaining reset INSIDE spawn_enemies() itself (changing the function's long-established contract and collapsing the 'two places must agree' lesson into one place for GameMaker only) before being corrected to keep spawn_enemies() creating instances only, with enemies_remaining staying split across start_game() and obj_bullet's win-check else branch exactly as Lesson 11 originally built it. The Slide Deck's Rule 10 demo-engine reference (and its spawn_wave()/speed-already-exists content, both leftover from before this lesson's actual GameMaker-featured, GameMaker-needs-a-new-speed-variable design) was corrected to GameMaker throughout, the Teacher's Script's matching spawn_wave() reference was fixed, and five speaker-notes entries were added (Rule 8, zero existing). Rule 9: a new generic Activity Diagram ("One Formula, Two Places - They Must Always Agree") and a GameMaker Visual node diagram (spanning all three places that change: spawn_enemies(), start_game(), and obj_bullet's win-check else branch) were built and wired into every engine's Handout and Key. While fixing this lesson, two bugs were also found and corrected in the just-shipped Lesson 13: a wrongly Title-Cased Scratch broadcast name ([Spawn Enemies] instead of the real lowercase [spawn enemies], introduced during the Lesson 13 fix pass itself) and a TIC-80 regression where that same Lesson 13 pass had incorrectly renamed TIC-80's correct spawn_wave() to spawn_enemies() and used the wrong sfx slot number (sfx(4), Lesson 8's life-lost sound, instead of sfx(6), the actual wave-clear

slot established in Lesson 12) - both corrected and reshipped before starting Lesson 14's own fixes.

Lesson 15 compliance pass: a stale obj_spawner reference, an abandoned Option-2 Unity field name, and the project's first process-only Activity Diagram: Lesson 15 ("Ready to Show," the Semester 1 capstone workshop-and-presentation day) got the same compliance treatment as Lessons 13 and 14, completing the three-lesson review Robert authorized while away from the keyboard. Structurally this lesson is unlike every other audit so far - it introduces no new mechanic, has no featured lecture-demo engine, and its Handouts/Keys are a self-assessment checklist and a 'known remaining quirks' troubleshooting reference rather than a worked Level 1 solution, so most of Rules 4/7/14 (which govern new code content) simply did not apply. Two genuine drift bugs were still found and fixed: GameMaker's quirks/diagnostics referenced a missing obj_spawner. prefix as a common last-mile bug, when the actual shared-state owner has been obj_gamemanager since the Lesson 9 compliance pass - corrected in both the Handout's engine notes and the Key's quirks list. Unity's quirks referenced roundAudioSrc as an example of a commonly-unassigned Inspector field - roundAudioSrc was the shared-sound-player field name from Lesson 12's abandoned Option 2 design (see the dedicated Option 2 preservation note), retired in favor of per-event fields when Robert chose Option 1; corrected to reference waveClip, one of the actual current per-event fields. Rule 10 did not apply (no lecture-demo engine this week, confirmed against the Plan's own "No lecture demo this week" format note), but the Slide Deck's two milestone-framing sections (Welcome Back, Milestone Moment) were each separately tagged 10 minutes despite the Teacher's Script and Plan agenda treating them as one combined 10-minute block - corrected to 5 minutes each so the on-screen pacing matches the actual schedule. Rule 8: zero existing speaker notes; five were added (slides 5, 7, 9, 11, 14), split across the Teacher's Script's five timed sections by best content match rather than rigid slide-boundary alignment, since the deck's own section structure (adding a dedicated "Milestone Moment" section) doesn't map one-to-one onto the script's five written sections. Rule 9: although this lesson teaches no new mechanic, the Standards call for an Activity Diagram in every lesson, so a new generic diagram - "Today's Workflow - Checklist, Polish, Present" - was built depicting the day's actual process (work the checklist, loop back through "Fix It First" if anything's broken, then tune/polish, then present) rather than any code logic, the first diagram in this project built around a class-day workflow instead of a programming concept, and wired into every engine's Handout and Key (no engine-specific node diagram applies, since no engine has new Level 1 code this week).

Lesson 12's Option 2 (shared sound-player design) - preserved reasoning for the book version: Recorded per Robert's explicit instruction, separate from the Option 1 implementation note above: even though Option 1 was chosen for the live course, the shared-sound-player idea the Lesson 12 content had already built (Godot's RoundSound swapping .stream between win/wave/lose clips; Unity's roundAudioSrc with PlayOneShot() layering three clips through one AudioSource) is a genuinely different, and genuinely teachable, architecture - not simply a mistake to discard. The case for reviving it in the book version: (1) it's a real-world pattern working game developers actually use - many shipped games route all short, mutually-exclusive one-shot UI/feedback sounds through a single pooled or shared AudioSource/AudioStreamPlayer rather than one dedicated node per sound, specifically to avoid the node/component count growing linearly with every new sound effect a game ever adds; (2) it's a natural, motivated moment to introduce the concept of a resource (an AudioClip/stream reference) being swapped at runtime rather than fixed at design time, which dedicated per-event nodes never require a student to think about; (3) PlayOneShot() (Unity) and a stream-swap-then-play() (Godot) are each worth teaching in their own right as engine-idiomatic answers to 'how do I play one of several interchangeable sounds from one player,'

independent of whether this specific lesson's three sounds (win/wave/lose) technically need it; (4) it would make a clean point of comparison against Scratch and GameMaker's simpler per-branch sound calls, turning the cross-engine contrast itself into a teaching moment ('two valid architectures for the same problem, here's the tradeoff') rather than something to quietly reconcile away. The case against reviving it, for balance: it does contradict the dedicated-per-event pattern Lessons 6 and 8 spent two lessons establishing, so introducing it here reads as a late architecture change rather than a planned progression unless the book's Lesson 6/8 chapters are rewritten to set up the contrast deliberately in advance; it also adds genuine new vocabulary (resource swapping, PlayOneShot's layering semantics vs. .Play()'s replace semantics) in a lesson Robert designed to be lighter than most, which may not fit Lesson 12's intended pacing even in the book. If revived, the clean approach would likely be: keep Lessons 6/8 exactly as the live course now has them (dedicated per-event nodes, Option 1 style), then introduce the shared-player idea in Lesson 12 as a DELIBERATE architectural pivot with its own named rationale ('your game now has four round-ending sounds - here's a different way to organize that'), rather than retroactively implying Lessons 6/8 should have used it from the start. Not decided; purely preserved reasoning for whoever revisits this at book-writing time, per Robert's explicit request to keep it rather than let Option 1's implementation erase the alternative's thinking.

Robert's own Unity project reconciliation: a fourth engine with the ownership-reassignment pattern, plus two real incidental bugs: Following the same process already used for GameMaker, Godot, and Scratch, Robert reconciled his own Unity Lesson 12-level project against canonical, starting with Bullet.cs and Enemy.cs. His Bullet.cs matched canonical's flash-on-spawn visual effect (color swap + scale pulse via a flashTimer countdown, structurally identical to Lesson 3's Complete Level 3 solution) but used Color.orange instead of canonical's Color.yellow - confirmed by Robert to compile and work fine (not flagged further). More significantly, Robert's project had moved bullet/enemy collision ownership onto Enemy.cs (OnTriggerEnter2D, the hit sound, and GameManager.RegisterKill() all living there) rather than Bullet.cs, matching the exact same deliberate-exploration pattern already tracked above for GameMaker and Godot - now a fourth confirmed data point, and the first one found in Unity specifically. Separately, Robert's Bullet.cs also played its own fire/shoot sound in Start() via a private AudioSource field, which he confirmed is a deliberate ownership choice (not a miss) and asked to be preserved as-is rather than reconciled to canonical's Player-owns-the-shot-sound placement - tracked as raw material for the 'Doing It the Doers of Stuff Way' chapter rather than fixed. Both files were rewritten to canonical for the current course version per Robert's explicit instruction: Bullet.cs regained OnTriggerEnter2D (checking the Enemy tag, playing hitSound via AudioSource.PlayClipAtPoint, calling GameManager.RegisterKill(), destroying both objects) and kept the flash effect (reverted to canonical's Color.yellow) and the fire-sound-in-Start() exactly as Robert wanted it kept; Enemy.cs was stripped back down to movement, the off-screen escape check, and GameManager.RegisterEscape() only. Two real, incidental bugs were caught along the way and flagged to Robert (not yet confirmed fixed in his project): Enemy.cs's OnTriggerEnter2D was checking other.CompareTag("Player") instead of "Bullet" - meaning hit detection likely wasn't registering via that path at all, since nothing in the game's design has the Player touching an Enemy register as a hit; and Enemy.cs carried a stray, unused using UnityEditor.UI; at the top, which compiles fine in the Editor but would break a standalone player build - Robert confirmed it was already greyed out as unused and wasn't sure how it got there. Robert separately confirmed his actual Bullet prefab is tagged "Player" rather than "Bullet", explaining the CompareTag("Player") bug directly - likely a copy-paste-from-Player-prefab origin rather than a code mistake, with the fix being a Tag dropdown change on the prefab itself (and a quick check that the Enemy prefab is actually tagged "Enemy" too), not a code change. Also worth flagging separately (not part of this reconciliation, not yet fixed in the shipped course

materials): the shipped Lesson 13 Unity content incorrectly describes hit-detection as having always lived on 'the Enemy prefab's script,' when canonical Lessons 4-12 confirm it has always been Bullet.cs - missed during Lesson 13's own compliance pass, tracked here as a found-but-not-yet-fixed continuity bug in the live course materials.

CORRECTION: the delivered Bullet.cs rewrite dropped the fire sound it was supposed to keep: The note above states the fire/shoot-sound-on-Bullet placement was 'kept as-is' per Robert's explicit instruction - that was wrong. Claude said it would keep it, then actually delivered a rewritten Bullet.cs with the audioSrc field and its Start()-time .Play() call removed entirely, directly contradicting Robert's stated intent and the record Claude had just written. Robert caught this by its real symptom (fire and hit sounds both gone) rather than by the code review itself. Hit sound was a separate, smaller issue layered on top: hitSound is a new field on Bullet (moved from Enemy's old enemyHit field as part of the collision-ownership reconciliation), so even with correct code the Inspector slot for it was never assigned on Robert's Bullet prefab - the old enemyHit assignment on Enemy doesn't carry over to a differently-named field on a different object. Both fixed: audioSrc and its Start() call restored to Bullet.cs, and Robert still needs to drag his hit-sound clip onto the new hitSound slot. Worth remembering for future reconciliation passes: double-check that delivered code actually matches what was promised in the same message, not just that the promise sounds right.

SECOND CORRECTION: Robert's hit sound wasn't actually missing, and his own fire-sound-ownership call changed too: Following on directly from the correction above. Robert re-tested and found the hit sound was never actually gone - it was present but extremely quiet (had to turn his laptop volume up to hear it) - and separately decided, after thinking about it more, that he'd rather the fire sound live in the canonical place (Player's own script) after all, so cross-engine logic stays consistent; his original ownership argument is saved for the refactoring chapter instead of kept live now. Bullet.cs's fire-sound field and Start() call were removed again (this time deliberately, matching canonical, not by mistake). The quiet-hit-sound symptom turned out to be real and diagnosable: Claude's first suggested fix (bump the PlayClipAtPoint volumeScale argument) was wrong - Robert correctly identified that volumeScale is clamped 0-1, a percentage of the clip's own volume, not a boost, so it can never counteract 3D distance rolloff. The actual, confirmed-working fix is calling PlayClipAtPoint at Camera.main.transform.position instead of the hit location - which is exactly the pattern GameManager's own round-ending sounds already used, and which turned out to be a genuine, previously undocumented bug in the shipped Lesson 4 Unity materials (tracked separately as its own ToDo and Notes entry) as well as a real regression introduced into Lesson 13's HandleHit() Coroutine rewrite (hitSound was dropped from that function entirely - fixed and reshipped). Robert's own Bullet.cs's hitSound call was corrected to the camera-position pattern too. The canonical Player-side fire sound is still pending - Robert's actual fire/shoot script hasn't been reconciled yet, since PlayerMovement.cs (reviewed and fixed earlier) turned out to contain no shooting logic at all.

Robert's own Unity GameManager.cs reconciliation: one genuine gap (missing waveClip), one legitimate addition worth keeping: Continuing the file-by-file Unity reconciliation after Bullet.cs/Enemy.cs, Robert's GameManager.cs turned out to already match canonical closely at the Level 3 Stalemate-fixed architecture - Update()'s new-Input-System restart check, ResetGame(), the EnemySpawner field, and RegisterKill()/RegisterEscape() all lined up with the shipped Lesson 10/11 Key almost exactly (matching formatting/commented-out-old-code noise aside). One genuine gap: Lesson 12's entire addition, the waveClip AudioClip field and its AudioSource.PlayClipAtPoint() call on wave-advance, was missing outright. Because Robert's WaveCleared() (the Lesson 11 Stalemate-fix architecture) already

exists, ahead of where Lesson 12's own Level 1 instructions assume a project is (they show the call inline in RegisterKill()'s nested if/else, since Level 1 doesn't have WaveCleared() yet), the call was added inside WaveCleared()'s else branch instead, next to winSound - the correct integration point for a project already at this level of accomplishment, not a literal copy of the Level 1 snippet. Separately, Robert's Start() plays a gameStartSound via AudioSource.PlayClipAtPoint() - not canonical GameManager code, but not drift either: sfx_start.wav was shipped all the way back in Lesson 6's Assets folder explicitly as a general-use stinger, and Robert is using it exactly as intended. Recommended he keep it. The reconciled file also dropped a stray commented-out using Unity.VisualScripting.Antlr3.Runtime.Collections; at the top and several dead commented-out code blocks (pre-Stalemate-fix RegisterKill()/RegisterEscape() logic Robert had left in place rather than deleted) - cosmetic cleanup, not an architecture question.

HUD.cs as a live-classroom debugging technique - worth teaching on purpose, not just confessing to: Reconciling Robert's HUD.cs (otherwise an exact match for canonical's scoreText/livesText/waveText) turned up one extra field: a Text displaying GameManager.gameActive.ToString() as raw True/False. Robert confirmed it was a debug leftover - but specifically one he'd used deliberately in class to demonstrate a technique: when printing to the console (or another normal debugging method) is inconvenient to show a room of students, temporarily wiring a bit of internal state straight onto the visible HUD is a legitimate, fast way to make it observable to everyone at once. Worth promoting from 'thing Robert happened to leave in' to an actual named technique/sidebar - 'abusing the HUD for debugging' - since it's a real, reusable classroom move, not just an artifact of one lesson's build. The field itself was still removed from Robert's reconciled HUD.cs as debug-only, not meant to ship in the finished game.

Robert's own Unity PlayerMovement.cs reconciliation: two real behavior gaps from splitting one combined check into four: The last file in the Unity Bullet/Enemy/GameManager/EnemySpawner/HUD/PlayerMovement reconciliation sequence. Movement, the screen clamp, directional sprites, and the footstep blink/sound were all present and matched canonical's Lesson 2 Complete Level 3 solution in spirit, but Robert's version checked each arrow key in its own separate if block (each calling a shared ImWalkingNow() helper) rather than canonical's structure of computing h/v first, then doing blink/sound/sprite exactly once inside a single combined moving check. Two real, if subtle, behavioral differences fell out of that structural choice: holding two direction keys at once (diagonal movement) calls ImWalkingNow() twice in the same frame, and blinkTime += Time.deltaTime isn't guarded against that, so the blink rate quietly speeds up only during diagonal movement; and there was no walkingBeep.Stop() anywhere - canonical explicitly stops the AudioSource when idle, but Robert's version only ever calls .Play(), so if that AudioClip loops (likely, for a continuous footstep beep), the sound keeps looping forever after the player stops moving. Reconciled back to canonical's single-combined-check structure, which fixes both. Two legitimate keepers, not reconciled away: named xBoundary/yBoundary fields (8f/4.5f) instead of canonical's hardcoded literals - a genuine readability improvement; and sr wired via a [SerializeField] private Inspector reference instead of canonical's GetComponent<SpriteRenderer>() call in Start() - a valid alternative wiring style, same SerializeField-vs-public question already tracked as its own ToDo. Robert still needs to re-drag the Inspector field references and playtest before confirming this version works in his actual project - not yet verified at the keyboard.

Unity reconciliation closed out: Player.cs's fire sound, the last piece: Robert had missed his own Player.cs in the file list - once found, it turned out to already match canonical almost exactly (fireCooldown decrement, space-key check, Instantiate, 0.2s cooldown reset). One pre-

existing, already-tracked deviation: Player.cs spawns the bullet offset 0.7 units above the Player's own position (bulletYOffset, a named constant - nicer than the magic numbers used in his other engines' projects) rather than exactly on it, the same cross-engine-consistent deliberate offset already logged above for Scratch/GameMaker/Godot - now confirmed as a FOURTH engine with the same personal habit. The only real gap was the fire sound itself, entirely missing (no AudioSource field at all) - added as a SerializeField AudioSource, with audioSrc.Play() called right next to Instantiate(), exactly where canonical puts it. This one doesn't need the Camera.main.transform.position treatment hitSound needed - a persistent, Inspector-added AudioSource defaults to 2D spatial blend, so it was never subject to PlayClipAtPoint()'s 3D distance-rolloff problem in the first place. This closes out the full Unity reconciliation sequence: Bullet.cs, Enemy.cs, GameManager.cs, EnemySpawner.cs, HUD.cs, PlayerMovement.cs, and finally Player.cs, all now matching canonical per Robert's own assessment ('mostly' - the few remaining differences being his own confirmed, deliberate keepers: the bullet-spawn offset and the [SerializeField] private vs. public fields question, both already tracked above).

Lesson 13 Unity fix: hit detection had actually drifted onto Enemy's script, not just been mis-described there - fixed by restoring Bullet-owns-detection and giving Enemy its own delayed-removal Coroutine: The previously tracked 'continuity bug' (Lesson 13's text calling the hit-detecting script 'the Enemy prefab's script') turned out to be worse than a wording slip once actually checked: the shipped level1Code genuinely had OnTriggerEnter2D, checking other.CompareTag("Bullet"), living on Enemy.cs - a real architectural regression from the Bullet-owns-detection rule this course has enforced since Lesson 4 ('only Bullet's OnTriggerEnter2D calls GameManager.RegisterKill()'). Fixed by moving hit detection back onto Bullet.cs, unchanged in principle, while keeping the genuinely new idea this lesson needs - a delayed, flashing removal - on Enemy, where it belongs: Enemy gets its own public IEnumerator HandleHit() Coroutine (Instantiate explosion, swap flash sprite, yield 0.1s, Destroy self), and Bullet's OnTriggerEnter2D starts THAT Coroutine on the Enemy instance itself (enemy.StartCoroutine(enemy.HandleHit())), not on Bullet - a Coroutine always runs on whichever MonoBehaviour StartCoroutine() is called on, so starting it on Bullet would cancel it the moment Bullet destroys itself a line later. This also surfaced and fixed a second, previously unnoticed problem: once the Enemy no longer disappears instantly, it sits on screen collidable for the full 0.1 seconds, which meant a second overlapping bullet could double-trigger the same still-flashing enemy and double-register a kill - the original shipped content's own guard (destroying the bullet immediately) never actually prevented this, since it only stopped the SAME bullet from re-triggering, not a different one. Fixed with other.enabled = false, disabling the Enemy's Collider2D the instant it's hit, before the Coroutine even starts. Rebuilt, PDF-verified, and reshipped to the Lesson 13 folder - checked in alongside Robert's own Lesson 13 code validation.

Lesson 13 retrofitted with the new-code highlighting standard, across all 8 engines: Robert asked for Lesson 13's Handouts and Keys to be recreated with the yellow-highlight-new-code convention (lines starting with "@@ " render on a yellow background with a legend line, established during the Lessons 8-10 compliance passes but never retrofitted into Lesson 12, 13, 14, or 15's build scripts - all four still used the older, unmarked code() helper). Checked and confirmed: Lessons 1-7 never had it either (it simply didn't exist yet), Lessons 8-10 use it heavily, Lesson 11 uses it only lightly (4 occurrences), and Lessons 12-15 have zero - so this is a real, previously unnoticed gap in the retrofit scope, not unique to Lesson 13. For this pass, Lesson 13's code() helper was updated to the highlighting version (copied from Lesson 10's), and every one of the 8 engines' level1Code blocks was hand-marked line by line: genuinely new or changed lines this lesson get @@ (the explosion-spawn/flash-swap/wait-or-delay lines,

Unity's other.enabled = false/Coroutine-on-Enemy lines, GameMaker's new Alarm 0 event, TIC-80's flash-countdown update loop, etc.); lines that only moved position but kept identical text (Scratch's "delete this clone", Godot's queue_free()) were deliberately left unmarked, matching the precedent set by Lesson 10's own code - the convention marks new TEXT, not relocated-but-unchanged text. Rebuilt, PDF-verified (text-checked across all 8 engines, visually spot-checked on Unity's rendered highlighting), and reshipped to the correct Handouts/ and Keys/ subfolders.

New-code highlighting retrofit completed for Lessons 12 and 14; Lesson 15 confirmed not applicable: Robert asked for the same new-code yellow-highlighting retrofit done on Lesson 13 to be extended to Lessons 12 and 15. Lesson 12 (the Sound lesson) and Lesson 14 (the wave-scaling-formula lesson) both got their code() helper updated to the highlighting version and every engine's level1Code hand-marked - Lesson 12's pattern across all 8 engines was consistently simple, since the lesson adds exactly one genuinely new sound (the wave-clear cue) per engine while every other sound line shown is pre-existing/optional-becoming-mandatory content, left unmarked per the precedent already set by Lesson 11's own winSound handling (new state/structure gets highlighted, pre-existing code shown for context does not, even inside a newly added branch). Lesson 14's pattern was the inverse - nearly every displayed line IS the week's change (the 5-to-(4+wave) formula, reused in 2-3 places per engine), so most lines in each code block ended up marked. A genuine, unrelated bug was caught and fixed along the way in 001 Game Creator's Lesson 14 content: the ResetGame code literally still read enemies_remaining, Set, 5, contradicting its own instructions to use 4 + wave - corrected to match. Lesson 15 was checked and confirmed to have no code() helper and no level1Code blocks at all - it's the Semester 1 capstone workshop day, introducing no new mechanic, built as a self-assessment checklist rather than worked code, so the highlighting rule genuinely has nothing to apply to there. All three lessons' Handouts and Keys rebuilt, PDF-verified, and reshipped to their correct Handouts/Keys subfolders.

Confirmed Unity gotcha: "Reload Scene only" leaves gameActive (and any other static) stuck from the previous Play session: Robert hit a real, reproducible bug in his own Unity project: enemies would spawn but never move, intermittently, only after a session that had ended in an actual win or loss, and only fixable (for the rest of that Editor session) by playing through to a win/loss and pressing R. Diagnosed collaboratively: gameActive is only ever reset to true inside GameManager.cs's ResetGame(), never inside Start(). With Unity's "When entering Play Mode" setting (Edit > Project Settings > Editor > Enter Play Mode Settings) set to "Reload Scene only" - a legitimate, common choice for faster Editor iteration, not a mistake - the scene's GameObjects ARE destroyed and recreated fresh each Play session (so Start() does run), but static fields are NOT reset (no domain reload), so gameActive quietly carries over whatever value the previous session left it at. A previous win/loss leaves it false; the next session's Enemy.cs then bails out of Update() immediately via if (!GameManager.gameActive) return; before ever reaching movement - a silent full freeze, not a slowdown, confirmed by Robert as the actual mechanism once he found the exact setting (shown in current Unity versions as a single "When entering Play Mode" dropdown with a "Reload Scene only" option, not the older two-checkbox UI; the dropdown's four exact options, confirmed directly from Robert's own Editor: "Reload Domain and Scene" - the default, full reload; "Reload Scene only" - Robert's setting, the one that causes this bug; "Reload Domain only"; and "Do not reload Domain or Scene" - the most aggressive speed-up, which would mask this specific bug differently, since Start() wouldn't even re-run). This is almost certainly also the reason Robert had a gameActive debug field on his HUD at one point (see the matching HUD-debugging-technique note above) - he'd already worked around this bug once before, just didn't remember how, and it resurfaced once that debug field was removed during reconciliation. Fix: add

gameActive = true; to Start(), tracked as its own ToDo for the shipped Lessons 10-12 materials above.

Robert's own marker, for himself: Lesson 12 is the Git checkpoint where his experimental Unity code was deliberately reconciled back to [mostly] canonical: Robert asked for this recorded explicitly, mainly as a personal note rather than course content: the reconciliation pass that just finished (Bullet/Enemy/GameManager/EnemySpawner/HUD/PlayerMovement/Player) is the point where his own Unity project's accumulated experimentation - the bullet/enemy collision-ownership exploration, the fire-sound-placement question, the gameActive HUD debugging technique, the SerializeField/public style question, assorted dead/commented-out code - was deliberately stripped back out and the course code returned to [mostly] canonical, specifically so he can find his own pre-reconciliation experimental code again later in Git if he wants to revisit any of it (for the refactoring chapter, or just out of curiosity) without having to reconstruct it from memory. Treat this entry as a locator/bookmark, not a course-content item - the actual content of what was reconciled is already captured in the matching Notes & Commentary entries above.

Lesson 13's Teacher's Key brought up to Rule 14 (complete code for all three levels) - the gap turned out to be bigger than reported: Robert reported the Key was missing worked solutions for Level 2/3. Investigation found the actual gap was larger: build_lesson13_handouts_keys_checklist.js's keyData array had never been wired to pull level1Code from handoutData at all (unlike Lesson 8's reference implementation), so the Key was rendering NO code block whatsoever - not Level 1, 2, or 3 - despite its prose (level1/why/mistakes bullets) reading as if it had one, which is almost certainly what made the gap easy to miss on a skim. Fixed by porting Lesson 8's full Rule 14 machinery: a keyData.forEach wiring step that copies level1Code from the matching Handout entry, a keyLevel2Default generator (Lesson 13's Level 2 is verification-only across all 8 engines - no new code, just confirming the flash/explosion/timing already works), and a keyLevel3 object with real, complete, working flicker-stretch code for all 8 engines, built on top of each engine's Level 1 code with @@ marking the Level 3-specific additions. buildKey() updated to render both new sections, matching Lesson 8's exact layout ("Level 2 - complete code:" / "Level 3 - complete code (all three goals combined):"). Verified via PDF text extraction that all 8 engines' Keys now show Level 1, 2, and 3 code blocks; rebuilt and reshipped.

Correction to the above: Lesson 13 Key's Level 3 code was only combining one of each engine's two code-bearing stretch goals: Robert caught that the Unity Key's Level 3 code showed only the flicker (alternating flashSprite/normalSprite) and not the second Level 3 idea from the Handout - scaling the explosion prefab over its lifetime (Time.deltaTime). The header text ("Level 3 - complete code (all three goals combined)") promised all of a lesson's Level 3 goals combined, but the first pass had only ever written the flicker goal for every engine, silently dropping each engine's second, equally concrete stretch idea (Godot: Tween-scaling the explosion; GameMaker: growing obj_explosion's image_xscale/yscale; GDevelop: a scale Tween on Explosion; TIC-80: cycling the explosion through multiple sprite frames; 001 Game Creator: a second, brighter Graphics variant swapped in via Timer; Unreal: fading BP_Explosion's opacity via a second Timeline track). Each engine's keyLevel3 entry was rewritten to combine both code-bearing goals (the third goal in each engine's Level 3 list is always a "notice this object needs no help from score/lives/waves" structural observation, which needs no code and was correctly left out). Verified via PDF text extraction that both the flicker and the explosion-animation code now appear in all 8 engines' Keys; rebuilt and reshipped.

New GameMaker gotcha found live in Robert's own project: image_blend is a persistent instance property, independent of sprite_index, and keeps applying to whatever sprite is

currently shown: While building his own GameMaker Script project, Robert added a personal extension beyond the lesson - alternating enemy colors at spawn via image_blend (half c_white, half a dark red make_color_rgb(200, 25, 55)) - and then hit the Lesson 13 hit-flash code exactly as shipped (other.sprite_index = spr_enemy_flash; with image_blend untouched). Symptom: the white-tinted enemies flashed correctly (coincidentally, since multiplying by white is a no-op), but the dark-red-tinted enemies flashed to a lighter version of their own color instead of true white. Root cause, diagnosed collaboratively: image_blend is not tied to sprite_index at all - it is a persistent instance property that GameMaker multiplies against whatever sprite is currently showing, every frame, until explicitly reassigned. Swapping sprite_index to spr_enemy_flash (a sprite painted white specifically so it shows true white untinted) does nothing to clear a leftover image_blend value, so that sprite still gets tinted by whatever color the enemy spawned with. This is the same underlying multiply-tint trap the lesson already warns about for a single flash color, just compounding once a student adds their own per-instance tinting on top of it. Added as a teacher-only code comment plus a new Mistakes bullet in Lesson 13's GameMaker Key (NOT the Handout, since this only applies to students who've gone beyond the lesson with their own image_blend-based color variation) explaining the fix: also set other.image_blend = c_white; alongside the sprite swap; and, if a student later adds the Level 3 flicker (toggling back to the enemy's normal sprite mid-flash), they'd need to save/restore the enemy's original image_blend value too, since the normal frame needs the real tint back. Rebuilt and reshipped; Handout confirmed unaffected.

GameMaker sprite-origin design note (for the book): centered origin is the standard convention for a top-down mover, not a counter-culture change to the engine's default: Robert hit a second, related artifact while testing Lesson 13's Level 3 growing-explosion stretch goal in his own GameMaker project: the explosion appeared to "travel" as it scaled up via image_xscale/image_yscale. Diagnosed as a sprite-origin problem - with the default top-left origin, GameMaker grows a scaled sprite by adding pixels only to the right and below the origin point, so the visible shape's apparent center drifts down-and-right as it grows, reading as movement. Setting the Explosion sprite's origin to Middle Center fixes this correctly (symmetric growth from the center), but then exposed a SECOND, compounding issue: with the explosion's origin now centered but obj_enemy's origin still at its top-left-corner default, spawning the explosion at other.x/other.y (the enemy's top-left corner, not its visual center) made the explosion appear up-and-left of the enemy. Robert asked whether to just center both objects' origins instead of patching around it, and whether doing so goes against some GameMaker cultural preference for the top-left default. Researched via web search: no such preference exists for moving/rotating/scaling objects - GameMaker's own default is purely a mechanical import default, kept mainly because it suits UI elements, HUD bars, and tile/background art that are naturally placed by a corner rather than a center. Multiple tutorial/course sources (a GameMaker Studio 2 Udemy course description, a Godalming College intro tutorial, GameMaker's own Sprite Editor documentation) consistently teach centering the origin specifically for characters and anything that rotates (image_angle) or scales, which describes this course's top-down enemies and effects exactly; the one documented exception is platformers, where a bottom-center origin is often preferred so a character's feet align with floor collisions - not relevant to a top-down shooter. Conclusion for the book retrofit: recommend centering BOTH the moving/scaling actor's origin AND any effect spawned relative to it, as the default teaching approach from whichever lesson first introduces rotation or scaling (a stronger candidate than Lesson 13, since Lesson 13's own Level 1/2 content never scales or rotates anything - this only surfaces once a student reaches a Level 3 stretch goal). Flagged concretely, since changing origin changes what x/y mean for that object and ripples into other code: any instance_create_layer spawn coordinates calibrated against the old corner-anchored position (they'll need retuning, since the same numbers will now place the object's center rather than its

corner), any escape/boundary check comparing an enemy's y against a screen edge (tuned against the sprite's top edge touching that line, not its center), and any effect-spawn code written to compensate for a corner-anchored parent (which becomes simpler, or unnecessary, once the parent's own origin is centered). Added as a teacher-only comment in Lesson 13's GameMaker Key, attached to the Level 3 growing-explosion code, covering the recommendation and the retuning checklist; flagged here for the book version as a design-convention note to adopt proactively rather than something to re-discover lesson by lesson. References consulted: gamemaker.io/en/tutorials/sprite-editor (GameMaker's own Sprite Editor documentation); online.godalming.ac.uk's "Catch The Clown" intro tutorial (Godalming College), which has students click Center for a character sprite; and the course description for "Become a Game Maker with GameMaker Studio 2" (Udemy), which ties centered origins to image_angle rotation and notes platformers as the common bottom-center exception.

New Lesson 13 Sidebar added: "What Is an Alarm, Actually? (And Why '6' Isn't Seconds)": Robert said he was genuinely unclear himself on what other.alarm[0] = 6; actually does in GameMaker, and asked for a sidebar on it - the same self-contained "textbook sidebar" format already used elsewhere in Lesson 13 (and Lessons 2, 3, 5, 7-10, 14). Researched and confirmed against GameMaker's own manual (manual.gamemaker.io's alarm instance-variable reference and its Set Alarm Drag-and-Drop reference) before writing: every instance has 12 built-in alarm slots (alarm[0] - alarm[11]), each tied to its own Alarm Event; the countdown is measured in STEPS (one step = one frame at the room's speed setting), not seconds - the detail Robert specifically flagged as the piece he'd been missing; GameMaker decrements every running alarm by 1 each step (checked after Begin Step, before the regular Step event), and the instant it hits exactly 0, the matching Alarm Event fires that same step, but only if that event actually exists on the object (ties directly into the Key's existing "forgetting to add the Alarm 0 event" Mistakes bullet); after firing, the alarm keeps counting down to -1 and sits there as its genuine "not running" idle state, which is why alarm[0] > -1 is the idiomatic "is this active" check rather than comparing to 0. The sidebar also ties the number 6 back to the lesson's own cross-engine timing: at a common 60-steps-per-second room speed, 6 steps works out to roughly the same ~0.1 second pause Godot's await and Unity's yield return use for the identical hit-flash delay - and flags that changing a project's room speed changes what the same alarm[0] = 6 line actually produces in real time, a genuine GameMaker-specific gotcha. Added as a second sidebar entry in Lesson 13's existing Sidebars document (alongside "The White Flash That Wasn't"), rebuilt, verified via PDF text extraction, and reshipped; the other 8 lessons' Sidebar files were rebuilt as a side effect of running the shared build_sidebars.js script but are byte-for-byte unchanged, so only Lesson 13's file was actually reshipped.

Genuine, multi-part Godot regression found and fixed in the shipped Lesson 13 Handout and Key: a phantom group check, a queue_free() scoping bug, and silently dropped mandatory hit-sound code: Robert reported the shipped Lesson 13 Godot code checks area.is_in_group("bullets") - a group that was never created anywhere in Lessons 1-12 - and pasted his own, correct working code as a comparison. Investigating turned up more than the one reported issue: (1) the phantom group check itself, which would silently always fail and mean the Enemy never responds to being hit at all if actually run as shipped; (2) the Level 1 code was built on a completely wrong function name, _on_area_entered, instead of the established _on_hitbox_area_entered used consistently since Lesson 4; (3) it called area.queue_free() instead of area.get_parent().queue_free() - a genuinely easy mistake, since area IS the bullet's own Hitbox child Area2D, not the Bullet scene's root, so this would destroy only the Hitbox and leave the actual Bullet (sprite, movement, everything) alive and uncollidable on screen forever; and (4) most significantly, the entire mandatory-since-Lesson-12 hit-sound block (hide(), $Hitbox.set_deferred("monitoring", false), $Hit.play(), await $Hit.finished - all made

mandatory baseline by Lesson 12's own Handout text, "confirm its own AudioStreamPlayer2D plays sfx_hit.wav when destroyed... add it now if it's missing") had been silently dropped from Lesson 13's "revised" code entirely, rather than correctly carried forward and adapted. The real design tension Lesson 13 needed to resolve, and hadn't: Lesson 12's hide()-immediately technique (so the Enemy can keep playing its hit sound invisibly before queue_free()) directly conflicts with Lesson 13's new flash effect, which needs the Enemy to stay VISIBLE. Fixed by removing hide() (a genuine, deliberate reversal, called out explicitly in both the Handout and Key rather than silently), keeping $Hitbox.set_deferred("monitoring", false) and $Hit.play() as the still-needed double-hit guard and sound trigger, and replacing await $Hit.finished with the lesson's own new await get_tree().create_timer(0.1).timeout - which still keeps the Enemy alive long enough for a short hit sound to finish, just without the explicit protection against a longer one, a tradeoff now documented in both files' prose. Corrected across: the Handout's level1 prose, level1Code, level2, and trouble arrays; the Key's level1 prose, why, talkingPoints, and mistakes arrays (two new Mistakes bullets added specifically covering the queue_free scoping trap and the phantom group check, since both are plausible genuine student mistakes even though they originated here as shipped-content bugs); and the Key's Level 3 flicker code (keyLevel3.Godot), which had the identical bugs baked into its own combined-code block. Rebuilt, verified via PDF text extraction that no _on_area_entered/is_in_group references remain outside the explanatory Mistakes bullet, and reshipped both the Handout and Key.

Second Godot Lesson 13 gap found while Robert built the Level 1 fix live: the explosion scene's own script, and its Timer's Autostart, were never actually specified anywhere in the content: After the regression fix above, Robert tested Level 1 and hit a new symptom: explosions spawn but never disappear, piling up on screen. Two real content gaps surfaced, not one: (1) the Setup steps told students to set the explosion Timer's Wait Time and One Shot, but never mentioned Autostart - without it, the Timer never begins counting down on its own, so timeout never fires, confirmed as part of the cause once Robert clarified he'd misread "One Shot On" as already covering it; (2) more significantly, Robert clarified "there is no explosion script - the changes were made to enemy.gd" - the content's own wording ("on the explosion scene's own script, connect the Timer's timeout signal...") implicitly assumed a script already existed on the explosion scene, without ever having a Setup step that says to actually create one and attach it. A screenshot of Robert's scene tree also surfaced a related structural trap: he'd built Explosion (wrapper root) > ExplosionSprite > Timer, nesting Timer two levels deep rather than as a direct child of the root - $NodeName in GDScript only resolves direct children, so even a correctly-scripted explosion would fail to find $Timer in that structure. Fixed by rewriting the Setup steps to be explicit and sequential: make the Sprite2D itself the scene's root (not a separate wrapper node), add Timer as its direct child with Wait Time/One Shot/Autostart all spelled out, then explicitly attach a new script to that root and write its _ready()/_on_timer_timeout() functions, shown in full for the first time (previously the explosion's own script was described in prose only, never actually shown as code). The full explosion-script code was added to the Handout's level1Code (auto-copied into the Key), the Key's Level 3 code was updated to carry the same _ready()/_on_timer_timeout() structure instead of assuming a script already existed, and a new Mistakes bullet plus a new Handout troubleshooting bullet were added covering both root causes (Autostart, and the missing/misattached script) plus the node-depth trap. Robert confirmed the corrected Level 1 steps worked once he rebuilt the node following this guidance. Rebuilt, verified via PDF text extraction, and reshipped both the Handout and Key.

Four genuine problems found in Scratch's Lesson 13 content - two were real architecture bugs, not just unclear wording: Robert raised four issues while reviewing Scratch's Handout: (1) the Explosion sprite's "go to x/y of Enemy" instruction never actually said which block to use;

(2) the Level 3 flicker's 0.03-second waits were too fast to perceive, 0.1 seconds tested better; (3) the Level 3 costume-cycle idea said "next costume" with no instructions for actually creating the extra costumes it needs; (4) why does the flash use "set [color] effect" instead of a sprite/costume swap the way a white-sprite fix would work elsewhere. Investigating (1) and (4) turned up real bugs, not just gaps. On (1): the Enemy is clone-based (an established fact from this course's own Lesson 5 Key), and Scratch broadcasts carry no data - so "go to [Enemy v]" would always go to the Enemy TEMPLATE sprite's own position, never the specific clone that was actually hit, meaning every explosion would appear in the same wrong spot regardless of which enemy died. Fixed with the standard Scratch technique for passing data through a broadcast: two new shared variables, hit_x/hit_y, set from the clone's own x position/y position immediately before broadcasting, read by Explosion's script instead of targeting the Enemy sprite directly. On (4): confirmed via web search that Scratch's "color effect" is a hue ROTATION around the color wheel, not a brightness/whiteness blend the way modulate/image_blend/spriteRenderer.color are in the other engines - "set [color] effect to 100" was never capable of producing a white flash on the enemy's red sprite at all, it would just shift its hue to some other color entirely. This directly validated Robert's instinct that a sprite/costume swap was the obviously correct approach. Fixed by importing enemy_invader_white.png (the same asset the other sprite-swap engines already use) as a second costume on the Enemy sprite itself, and switching the flash mechanism from "set [color] effect to 100" to "switch costume to [flash]" throughout Level 1, Level 3, the Handout, and the Key - also requiring a Setup step to add the costume and name both costumes clearly, and changing the spawn-reset step from "set [color] effect to 0" to "switch costume to [normal]". (2) and (3) were content-clarity fixes: Level 3's flicker timing changed from 0.03s to 0.1s waits per Robert's own testing, and a concrete costume-creation step (duplicate the existing costume in the Costumes tab, edit each copy in the Paint Editor) was added to the Level 3 prose and referenced in the Key's Level 3 code comments. Updated: the Handout's setup, level1 prose, level1Code, level2, level3, and trouble arrays; the Key's level1 prose, why, talkingPoints, and mistakes arrays; and keyLevel3.Scratch's combined code. Worth flagging for the book pass: Lesson 5's own sidebar already correctly described Scratch's color effect as a hue shift rather than a tint, which makes this Lesson 13 bug a case of the course's own documented understanding not actually being applied when the Scratch flash code was first written - worth a cross-check of any other Scratch "set [color] effect" usage elsewhere in the course for the same mistake. Rebuilt, verified via PDF text extraction, and reshipped both files.

Scratch's Level 3 explosion idea simplified: Robert's size-growth suggestion replaces costume-cycling as the primary recommendation: Robert questioned why the Level 3 explosion animation needed new costumes at all, when "set size to ()%" stepped up across the same delays already used for the flicker (105%, then 110%, then 115%) looked just as good in his own testing - no extra art required. This is a genuine improvement, not just a simplification: every other engine's Level 3 explosion idea this lesson is already an EXPANDING-SCALE animation (Godot's Tween, Unity's Time.deltaTime growth, GameMaker's image_xscale/yscale, GDevelop's Tween, Unreal's Timeline), so Scratch's costume-cycling idea was actually the one outlier using a different technique for the same goal - Robert's suggestion brings Scratch into line with the other seven. Made size-growth the primary recommendation in the Handout's Level 3 prose (costume-cycling kept as a secondary, optional "if you'd rather hand-animate" mention, still with its creation instructions, since it remains a legitimate technique for a student who wants genuinely different frames rather than a single scaling image) and swapped the Key's demonstrated Level 3 code from the costume-cycle version to the size-growth version. One real catch worth keeping in the Key: Explosion is the SAME sprite reused on every hit, not a fresh clone, so without an explicit "set size to (100) %" reset at the start of each explosion, size would carry over and compound across a round (each explosion starting bigger than the last) - this

reset is included and called out in the Key's code comment. Rebuilt, verified via PDF text extraction, and reshipped both files.

Canonical Scratch Level 3 code updated to match Robert's own tested build, plus a new Mistakes bullet and a real double-kill bug flagged for the book: Robert shared two screenshots of his own working Level 3 Scratch project - Explosion's scripts and Enemy's full touching-Bullet branch - and asked for the canonical Key to be updated to match what he'd actually built and tested, rather than what was previously shipped. Three real differences, all adopted: (1) Explosion gained a separate "when green flag clicked -> hide" script, more robust than the previous vague "hide it by default" Setup wording (which only meant toggling the Sprite pane's eye icon) - this was folded into Level 1 as well as Level 3, since a reliably-hidden Explosion at game start matters from the first hit onward, not just once Level 3's stretch goals are added. (2) The Level 3 flicker now uses three "next costume" blocks instead of three explicit "switch costume to [flash]"/"[normal]" blocks - Robert confirmed this works because Enemy's two costumes (normal, flash - the white enemy sprite imported back in Setup) are exactly two, in that order, so a fresh clone starting on normal just toggles back and forth each call. A new Mistakes bullet was added explaining the fragility this introduces: the toggle only works with exactly two costumes in that order, and breaks (cycling through all costumes instead of flashing) if Enemy ever gains a third. (3) Explosion's size-reset now happens before show/position ("set size to 100%" -> show -> go to x/y -> 105%...) rather than after, matching Robert's exact tested order - functionally identical either way since Scratch runs synchronously to the next wait/yield point, but canonicalized to match what he built. Updated: the Handout's level1/level1Code/level3 arrays, and the Key's keyLevel3.Scratch code block and mistakes array. Rebuilt, PDF text-verified against the screenshots line by line, and reshipped both files. Separately, Robert flagged a genuine bug he's hit occasionally in his own Scratch project, explicitly NOT to be fixed in the live course: a bullet traveling through two overlapping enemy clones can destroy both, but score/enemies_remaining only register one hit, recreating the Stalemate bug - traced to the Lesson 3 decision to let bullets keep traveling rather than destroying them on first hit, combined with no per-bullet or per-clone guard against a second overlapping touch before either clone finishes deleting itself. Logged as a book-only ToDo with candidate fixes (a per-bullet already_hit flag, or disabling each clone's own touching-check immediately on hit, the same pattern Unity and Godot already use for their own Lesson 13 double-hit guards) rather than changed now.

Revision Note

First entry, written the same week as the Unreal Lesson 2/3/4/13 rebuild and the first pass of Cross_Engine_Consistency_Reference.docx. Updated the same week to add Standards Rule 6 (SuperStretch/Refinement and Style/visual-scripting reuse) and scope, but not begin, the question of retrofitting it into Lessons 1-15. Updated again to add Standards Rule 13 (dedicated manager object for shared state), the Lesson 6 GameManager retrofit, and Lesson 7's full compliance pass (GameManager routing, Rule 11 naming-drift fixes, the new Rule 9 diagram set, and Unity Visual Scripting prose tightening). Updated again to add a GameManager mini-lecture to Lesson 7's live slide deck (placed at the start of the lab section, separate from the main lecture) plus the paired ToDo to relocate that discussion to Lecture 06 once the book manuscript reaches it. Updated again to add Lesson 8's own full compliance pass (the lives/has_lost GameManager retrofit, Rule 11/12 TIC-80 and Scratch naming fixes, Rule 7 visual-path explicitness fixes, a new Rule 9 diagram set with a newly-documented Handout-vs-Key placement convention for the three dual-track engines, and the Rule 10/Rule 8 slide deck fixes correcting a Godot-leftover demo reference and adding its first six speaker-notes entries),

and to narrow the Rule 7/8/9/13 retrofit ToDo items' remaining scope accordingly. Updated again to add Lesson 9's own full compliance pass (a four-engine Rule 13 GameManager retrofit for score - GameMaker, Scratch, Godot, and Unreal - plus a Rule 11/12 Scratch naming fix, a Rule 7 Unity Visual Scripting explicitness fix, a new Rule 9 diagram set reusing the established Handout-vs-Key placement convention, and the Rule 10/Rule 8 slide deck fixes correcting a Godot-leftover demo reference and adding its six speaker-notes entries), and to narrow the Rule 7/8/9/13 retrofit ToDo items' remaining scope to Lessons 10-15 accordingly. Updated again to record the GameMaker __dnd_lives/__dnd_score correction (Lessons 8-9) and five follow-up To-Do items (Lessons 10-15 review, obj.variable vs. Target drop-down, an If Lives comparison sidebar, GameMaker Prefabs, and dropping asset-name prefixes). Updated again to record the Unity static-members items, the cross-engine name normalization To-Do and the canonical bug-hunt decision, plus the Lesson 10 compliance pass and the cross-engine design-drift To-Do (HUD object, prefabs). Updated again at the Lesson 11 review to point to the new Reflex Habits design-notes document and to record the name "the Stalemate" for the win/lose gap bug and the Lesson 11 compliance pass (active engines only). Updated again to record a spitballed new companion course family - Debugging 101, Testing 101, and a combined Testing/Debugging 201 - raised by Robert as a sideways idea, not yet scoped. Updated again to record the GML Visual "no Else If block" gotcha (not yet audited against shipped content) and to extend Standards Rule 12's scope to cross-engine pop-up/HUD text presentation (centering, placement, size), with reasonable shared defaults proposed but not yet decided or audited. Updated again to record Lesson 12's compliance pass and its genuine sound-architecture conflict (a new shared sound-player design in Godot/Unity contradicting the Lesson 6/8 dedicated-per-event pattern, resolved for the live course as Option 1 - retire the shared player, reuse Lesson 8's sfx_gameover.wav instead of a new sfx_lose.wav asset), plus a separate note preserving Option 2's reasoning (the shared-player design) for reconsideration at book-writing time. Updated again to record Robert's book-retrofit decision to pull all scattered optional sound content out of Lessons 2-8 and consolidate it into a genuinely dedicated Lesson 12 (live course unaffected), plus the sharper Level 1-versus-Level 2 purpose framing it surfaced - Level 1 as a classroom-pacing safety valve, Level 2 as the real preferred target especially for self-paced study - with a leaning that Level 2 goals should exclude sound in the earlier lessons once consolidated. Updated again to record Robert's decision to abandon an ongoing Level 4 (refactoring/personal-style) concept in favor of a single dedicated final chapter, "Doing It the Doers of Stuff Way," delivered outside the 30-lesson cadence as a take-home capstone-adjacent lecture. Updated again to fully scope a new Lesson 15 Complete Codebase deliverable (active 5 engines, Level 3 code included via a Rule 14 backfill, GameMaker Visual represented as full per-object node-flow diagrams rather than prose) - explicitly deferred, not started, until the remaining lessons are reviewed. Updated again to record a GameMaker Lesson 12/wave_cleared() reconciliation note (Level 1 vs. the Lesson 10/11 Level 3 Stalemate-fix architecture), surfaced while Robert reconciled his own drifted GameMaker Script project - explicitly left as-is for now, earmarked for either a later explicit refactor or the planned final refactoring chapter. Updated again to record six candidate refactorings Robert spotted while bringing his own GameMaker project back into compliance (enemy-count self-reporting on Create, a named enemies-per-wave variable instead of a hardcoded count, consolidating enemies_remaining updates onto obj_enemy's Destroy event, moving the lives check off obj_enemy onto the player/manager, extracting player movement into its own script, and eventually retiring obj_spawner once its sole remaining job - a single start_game() call - moves onto obj_gamemanager's own Create event) - captured as raw material for the "Doing It the Doers of Stuff Way" chapter, not implemented now, with obj_spawner's own historical shrinkage across the course flagged as a good real example of incremental refactoring to walk through. Updated again to record a genuine GameMaker naming-drift fix: the drag-and-drop sound block is correctly called "Play Audio" (confirmed by Robert against his own GameMaker install), not

"Play Sound"/"Audio Play Sound" as several shipped Lessons 8, 10, 11, and 12 materials had it - corrected across the affected build scripts and reshipped, while Lesson 13's similarly-worded references were checked and confirmed to be 001 Game Creator's and Unreal's own correct, different terminology. Updated again to add a new known case to the Rule 12 reconciliation list: bullet/enemy collision ownership is opposite between GameMaker (bullet's Collision event destroys the enemy) and Godot (enemy's Hitbox Area2D destroys the bullet), found while Robert reconciled his own Godot project against the Lesson 12 code; tracked only, not unwound now by Robert's explicit decision. Updated again to record the spawn_enemies() ownership asymmetry (GameMaker: free-standing global Script resource; Godot: owned by main.gd, reached from GameManager via get_parent()) and its tangled two-way obj_spawner/main.gd non-parallel, plus Robert's request to extend the eventual Rule 12 pass to Scratch's objects/events/broadcasts as well, likely via explicit Data Dictionary cross-references rather than literal renaming. Updated again to record a Godot-only, deferred decision: whether signal connections should be written explicitly in code (.connect(), as the Keys currently show) or wired through the Editor's Signals tab (Robert's own project uses this instead), found while Robert reconciled his enemy.gd - tracked only for now, no cross-engine parallel. Updated again to record Robert's own cross-engine ownership-reassignment pattern in GameMaker and Godot (shared collision/sound logic moved to whichever object Robert personally felt should own it, while determining his own eventual refactoring preferences) as raw material for the "Doing It the Doers of Stuff Way" chapter rather than a design to keep, plus Robert's own deliberate and cross-engine-consistent bullet-spawn-offset deviation, to be noted now and removed for the book's canonical code later. Updated again, same session, to CORRECT that entry: Scratch is not an example of Robert's drift the way GameMaker and Godot are - Claude initially misread Scratch's Bullet-owns-nothing design as drift, but Scratch's own shipped Key deliberately moves collision-ownership from Bullet (Lessons 3-4) to Enemy (Lesson 5 onward) on purpose, documented in the Lesson 5 Key itself (Scratch clones have no unique ID, so only the touched Enemy clone can identify and delete itself) - a genuine third Rule-12 data point and a worthwhile Scratch-specific teaching point, not drift to reconcile away. Updated again to record Robert's confirmation, while reconciling his Scratch GameManager, that the self-reporting enemies_remaining pattern (refactoring candidate 1 from the GameMaker list) was independently built in Scratch too and is being reconciled back to canonical, plus a personal reminder to retest two Scratch boundary values (escape y and spawn y) at full screen rather than only the editor's inset window. Updated again to record Lesson 13's compliance pass - the worst drift found in any pass to date (stale obj_spawner/spawn_wave references in GameMaker and TIC-80, a Rule 4 GameMaker Visual block-choice gap, a Scratch broadcast-casing slip caught before shipping, and a real Unity/Godot timing bug where score/win-check fired after the hit-reaction delay instead of before it) plus the project's first Rule 9 Activity Diagram built in direct response to a student question, and a new GameMaker Visual node diagram showing an object gaining a brand-new event. Robert then granted standing authorization to audit and fix Lessons 14 and 15 to the same standard while away from the keyboard, to be reviewed together as completed work on his return. Updated again to record Lesson 14's compliance pass - a stale spawn_wave()-vs-spawn_enemies() naming drift affecting Scratch, Godot, and GameMaker (plus a near-miss Scratch broadcast-casing catch), a genuine Godot/GameMaker start_game() ordering bug between wave and enemies_remaining, a brief GameMaker architecture regression (enemies_remaining reset folded into spawn_enemies() itself) caught and reverted to the established two-call-site split, a new Rule 9 diagram pair, Rule 8 speaker notes, and a Rule 10 demo-engine correction (GameMaker, not Godot) - plus a genuine Rule 12 discovery that TIC-80 legitimately uses a different name (spawn_wave) than the other engines, found only because fixing the other four engines required confirming TIC-80 was not itself drifted. Two bugs accidentally introduced during the Lesson 13 fix pass were also caught and corrected while working on Lesson 14: a wrongly Title-Cased Scratch broadcast name, and a

TIC-80 regression (a wrong rename plus a wrong sfx slot number) - both reshipped before continuing. Updated again to record Lesson 15's compliance pass, completing the three-lesson Lesson 13/14/15 review Robert authorized while away from the keyboard: a stale obj_spawner reference (GameMaker, should be obj_gamemanager), a leftover reference to Lesson 12's abandoned Option 2 Unity field name (roundAudioSrc, corrected to the real waveClip), a slide-timing correction (two milestone-framing sections re-tagged from 10+10 minutes down to 5+5 to match the actual 10-minute combined block), five new Rule 8 speaker notes, and the project's first Activity Diagram built around a class-day workflow (checklist, fix, polish, present) rather than a programming concept, since this lesson introduces no new mechanic at all. All three lessons (13, 14, 15) are now fully audited, fixed, diagrammed, rebuilt, verified, and shipped, ready for Robert's review as completed work. Updated again to record Robert's "Sound Manager" proposal for the book version of Lesson 12: keep the scattered early Level 3 sound stretch goals in Lessons 2-8 (reversing the earlier strip-it-out leaning) and instead make Lesson 12 the lesson where a student's scattered sound gets explicitly refactored into one formal SoundManager object, reviving and sharpening the already-preserved Option 2 reasoning and matching the course's other extract-into-a-manager lessons (GameManager, spawn_enemies(), wave_cleared()). Updated again to record Robert's Unity Bullet.cs/Enemy.cs reconciliation - a fourth confirmed engine with the bullet/enemy collision-ownership-reassignment exploration pattern (added to the Rule 12 ToDo above), a deliberate fire-sound-on-Bullet placement choice kept as-is for the refactoring chapter, two real incidental bugs found and flagged (Enemy.cs checking CompareTag("Player") instead of "Bullet", traced to the Bullet prefab itself being tagged "Player"; and a stray, unused using UnityEditor.UI; that would break a standalone build), and a new tracked-but-not-yet-fixed continuity bug in the shipped Lesson 13 Unity materials (hit-detection wrongly described as always having lived on the Enemy's script). Updated again to record Robert's GameManager.cs reconciliation: one genuine gap (the Lesson 12 waveClip field/sound, missing entirely, added into the Stalemate-fix WaveCleared()'s else branch since his project is already past Level 1), and one legitimate kept addition (a gameStartSound played in Start(), using the sfx_start.wav asset shipped back in Lesson 6 for exactly this purpose). Updated again to record a new deferred-by-design ToDo: whether Unity's early lessons should teach [SerializeField] private over plain public fields, raised when Robert's own EnemySpawner.cs used the former while explicitly declining to request the same change in canonical - he wants this decided as its own intentional discussion, not defaulted either way, with a leaning toward keeping canonical on public fields for beginner simplicity. Updated again to record the close of the file-by-file Unity reconciliation: HUD.cs's one deviation (a GameManager.gameActive debug Text field) turned out to be a deliberate, reusable live-classroom debugging technique worth its own named sidebar - wiring internal state straight onto the visible HUD when the room can't easily see a console - not just an artifact to remove (removed from the reconciled file regardless, as debug-only). PlayerMovement.cs, the last file, had two real behavior gaps traceable to one structural choice (four separate per-key if blocks instead of canonical's single combined moving check): blinkTime could double-increment during diagonal movement, and walkingBeep.Stop() was missing entirely, meaning a looping footstep clip would never actually stop when the player went idle. Both fixed by reconciling to canonical's single-check structure; Robert's named xBoundary/yBoundary fields and Inspector-wired sr reference were both kept as legitimate improvements. Robert still needs to re-drag the Inspector references and playtest this file before it's confirmed working - not yet verified at the keyboard, unlike every other file in this reconciliation sequence. Updated again to record and correct a real error: the Bullet.cs rewrite Claude delivered had quietly dropped the audioSrc field and its Start()-time fire-sound .Play() call, directly contradicting Robert's explicit instruction to keep it and the note Claude had just written claiming it was kept - caught by Robert via the actual symptom (fire and hit sounds both missing in-game) rather than by review of the code itself. Fixed by restoring audioSrc/.Play() to Bullet.cs; separately, hitSound (moved from Enemy's old

enemyHit field) needs its own fresh Inspector drag onto Bullet's prefab, since it's a new field with no carried-over assignment. Updated again to record the actual resolution: the hit sound was never missing, just very quiet, and Robert correctly diagnosed why (PlayClipAtPoint()'s 3D distance rolloff, not fixable via the volumeScale argument as Claude first suggested) and the real fix (play it at Camera.main.transform.position, matching GameManager's existing sounds) - which turned out to be a genuine, previously undocumented bug in the shipped Lesson 4 Unity materials (marked for book-only correction per Robert's decision) and a real regression in Lesson 13's HandleHit() Coroutine rewrite (the hit sound had been dropped entirely - fixed and reshipped, including a new troubleshooting note and a new Rule-9-adjacent sidebar candidate for the rolloff gotcha itself). Robert also settled his earlier fire-sound-ownership question in the other direction from his first instinct: keep it in the canonical Player-side location after all, for cross-engine consistency, saving his own ownership argument for the refactoring chapter instead - Bullet.cs's fire-sound field was removed again, this time intentionally. The canonical Player-side fire sound itself is still pending Robert's actual fire/shoot script, since PlayerMovement.cs turned out not to contain any shooting logic. Updated again to record Player.cs - the file Robert had initially missed - closing out the Unity reconciliation sequence: it already matched canonical almost exactly (one already-tracked deviation, the bulletYOffset spawn offset, now confirmed as a fourth engine with the same personal habit), with the fire sound itself the only real gap, added as a SerializeField AudioSource with audioSrc.Play() right next to Instantiate(), exactly where canonical puts it - no Camera.main.transform.position fix needed here, since a persistent Inspector-added AudioSource defaults to 2D spatial blend and was never subject to PlayClipAtPoint()'s rolloff problem. Robert considers Unity now returned to [mostly] canonical across all seven reconciled files (Bullet, Enemy, GameManager, EnemySpawner, HUD, PlayerMovement, Player). Updated again to record Robert's request to recreate Lesson 13's Handouts and Keys with the new-code yellow-highlighting convention applied - a retrofit gap discovered along the way: Lessons 12, 13, 14, and 15 had all shipped using the older, unmarked code() helper, never updated after the highlighting convention was established during Lessons 8-10's compliance passes. Lesson 13 was fully retrofitted this pass, hand-marking genuinely new/changed lines across all 8 engines' code blocks (moved-but-unchanged text, like Scratch's relocated "delete this clone", was deliberately left unmarked, matching Lesson 10's own precedent); Lessons 12, 14, and 15 are now tracked as a new ToDo for the same treatment. Updated again to record Lesson 12 and Lesson 14's own new-code highlighting retrofits, completing Robert's follow-up request: Lesson 12's eight engines each needed only their one genuinely new wave-clear sound line marked, Lesson 14's eight engines needed nearly every line marked (the whole lesson is the 5-to-(4+wave) formula), and Lesson 15 was confirmed to have no code blocks at all - a capstone workshop day with nothing for the rule to apply to. A genuine, unrelated bug was also caught and fixed in 001 Game Creator's Lesson 14 ResetGame code, which still read enemies_remaining, Set, 5 despite its own instructions saying 4 + wave. Updated again to record a confirmed, reproducible Unity gotcha found in Robert's own project: gameActive is never reset in GameManager.cs's Start(), only in ResetGame(), which silently freezes all enemy movement (though not spawning) on any Play session following a win/loss, specifically when the Unity Editor's "When entering Play Mode" setting is "Reload Scene only" - diagnosed collaboratively over several messages, confirmed by Robert finding the exact setting, and tracked as both a ToDo (add gameActive = true; to Start() in the shipped Lessons 10-12 materials) and a troubleshooting-sidebar candidate, since it's a legitimate Editor setting choice, not a mistake. Updated again to record a genuine fix to the shipped Lesson 13 Unity materials: the previously tracked "continuity bug" turned out to be a real architecture regression, not just mis-described text - hit detection had actually moved onto Enemy.cs's OnTriggerEnter2D, breaking the Bullet-owns-detection rule enforced since Lesson 4. Fixed by restoring detection to Bullet.cs while giving Enemy its own delayed-removal Coroutine, started on the Enemy instance specifically (not Bullet, which would cancel it on self-

destruction) - which also surfaced and fixed a second, previously unnoticed double-hit bug (the Enemy no longer disappears instantly, so a second overlapping bullet could double-register a kill; fixed via other.enabled = false). Rebuilt, verified, and reshipped for Robert to check in alongside his own Lesson 13 code validation. Also added, per Robert's explicit request, a personal marker note (not course content) recording Lesson 12 as the Git checkpoint where his own experimental Unity code was deliberately stripped back out in favor of canonical, so he can find his pre-reconciliation code again later if he wants to revisit it. Updated again to record Lesson 13's Teacher's Key Rule 14 fix: what Robert flagged as a missing Level 2/3 worked solution turned out, on investigation, to be a Key with no code block at all (Level 1, 2, or 3), because its keyData array had never been wired to pull level1Code from the Handout the way Lesson 8's reference implementation does - fixed by porting that wiring plus a new keyLevel2Default verification-only note and a keyLevel3 object with real, complete flicker-stretch code for all 8 engines, rebuilt and reshipped. Updated again to correct that same fix: Robert caught that the Unity Key's Level 3 code combined only the flicker goal, silently dropping each engine's second Level 3 idea (an expanding/animated explosion) despite the section header promising all goals combined - every engine's keyLevel3 entry was rewritten to include both code-bearing stretch goals, rebuilt and reshipped. Updated again to record a new GameMaker gotcha found live in Robert's own project: image_blend is a persistent instance property independent of sprite_index, so his own per-enemy color variation (added via image_blend in spawn_enemies()) survived straight through the Lesson 13 hit-flash untouched, making the flash render as a lighter version of each enemy's own spawn color instead of true white for any non-white-tinted enemy - fixed with a teacher-only code comment and Mistakes bullet added to the GameMaker Key specifically (Handout confirmed unaffected), explaining that other.image_blend = c_white; must be set alongside the sprite_index swap. Updated again to record a related GameMaker sprite-origin design note, researched and confirmed via web search: a scaling object's apparent "travel" is caused by a non-centered origin, and centering both the scaling object's and its parent's origin is the standard, uncontroversial convention for a top-down mover/rotator/scaler (not a counter-culture change to GameMaker's default, which exists mainly for corner-anchored UI/tile art) - added as a teacher-only comment on Lesson 13's GameMaker Key Level 3 code, with a checklist of what else needs retuning (spawn coordinates, boundary checks, dependent effect-spawn code) if a teacher or student makes the change. Updated again to record a new Lesson 13 Sidebar, researched against GameMaker's own manual and written at Robert's request after he found he genuinely wasn't sure what other.alarm[0] = 6; does: alarms are 12 per-instance countdown timers measured in steps (not seconds, the detail Robert specifically missed), decremented automatically each step, firing their matching Alarm Event only if one exists, then idling at -1 until reset - added as a second sidebar entry in Lesson 13's existing Sidebars document, rebuilt and reshipped. Updated again to record a genuine, multi-part Godot regression caught by Robert in the shipped Lesson 13 Handout and Key: a phantom area.is_in_group("bullets") check (that group was never created anywhere in the course), the wrong function name (_on_area_entered instead of the established _on_hitbox_area_entered), a queue_free() scoping bug (area.queue_free() destroys only the bullet's Hitbox child, not the Bullet itself, via area.get_parent().queue_free()), and the complete silent loss of the mandatory-since-Lesson-12 hit-sound/double-hit-guard code, which needed to be reconciled against a real design conflict (Lesson 12's hide()-immediately sound technique directly fights Lesson 13's need to keep the Enemy visible for its flash) rather than just dropped. Fixed across the Handout's and Key's prose and code, including the Key's Level 3 combined code, which had the same bugs baked in; rebuilt, verified, and reshipped both files. Updated again to record a second Godot Lesson 13 gap, found live while Robert tested the fix: explosions spawned but never disappeared, traced to two real content gaps - the explosion Timer's Autostart was never specified in Setup (only Wait Time and One Shot were), and the explosion scene's own cleanup script was never actually shown or explicitly instructed to be

created/attached anywhere, only implied in prose - Robert had put the changes in enemy.gd instead, which has no way to reach back into a spawned explosion later. A screenshot also caught a node-depth trap (Timer nested two levels under an extra wrapper node, where $Timer can't resolve it). Fixed with explicit, sequential Setup steps, the explosion script shown in full code for the first time, and new Mistakes/troubleshooting entries; Robert confirmed Level 1 works now. Updated again to record four Scratch issues Robert raised, two of which were genuine architecture bugs rather than unclear wording: "go to x/y of Enemy" could never have worked correctly against a clone-based Enemy (fixed with shared hit_x/hit_y variables, the standard Scratch technique for passing data through a broadcast), and "set [color] effect to 100" was never capable of producing a white flash at all, since Scratch's color effect rotates hue rather than blending toward white (fixed with a genuine costume swap, confirmed via web search and consistent with the white-sprite-swap fix already used in the other seven engines) - plus a flicker-timing adjustment (0.03s to 0.1s, per Robert's own testing) and new costume-creation instructions for the Level 3 explosion animation. A new ToDo was added to audit the rest of the course's Scratch content for the same color-effect-as-tint mistake. Updated again to record Robert's own improvement to Scratch's Level 3 explosion idea: his tested "set size to ()% " growth approach (105%, 110%, 115%, same delays as the flicker) replaced costume-cycling as the primary recommendation, bringing Scratch in line with every other engine's explosion-scaling Level 3 idea rather than being the one outlier using a different technique - with a genuine catch caught and documented (Explosion is a reused sprite, not a fresh clone, so size needs an explicit reset each time or it compounds across a round). Add to this document as the course continues; do a dedicated consolidation pass every so often rather than continuously.